What Drives Representation Steering? A Mechanistic Case Study on Steering Refusal

Stephen Cheng and Sarah Wiegreffe* and Dinesh Manocha*
University of Maryland, College Park
Correspondence: scheng03@umd.edu†

Abstract

Applying steering vectors to large language models (LLMs) is an efficient and effective model alignment technique, but we lack an interpretable explanation for how it works–specifically, what internal mechanisms steering vectors affect and how this results in different model outputs. To investigate the causal mechanisms underlying the effectiveness of steering vectors, we conduct a comprehensive case study on *refusal*. We propose a multi-token activation patching framework and discover that different steering methodologies leverage functionally interchangeable circuits when applied at the same layer. These circuits reveal that steering vectors primarily interact with the attention mechanism through the OV circuit while largely ignoring the QK circuit. Freezing all attention scores during steering drops performance by only 8.83% across three model families. A mathematical decomposition of the steered OV circuit further reveals semantically interpretable concepts, even in cases where the steering vector itself does not. Leveraging the activation patching results, we show that steering vectors can be sparsified by up to 85-96% while retaining most performance, and that different steering methodologies agree on a subset of important dimensions.

1 Introduction

Aligning large language models to behave in accordance with human intent is a central challenge in deploying these systems safely (Anwar et al., 2024). Steering vectors have emerged as a lightweight model alignment technique that acts on the model's hidden activations at inference time (Zou et al., 2025). This approach has been applied across a range of alignment-relevant tasks, including reducing hallucinatory behavior (Chen et al., 2025; Rimsky et al., 2024), controlling persona and style (Subramani et al., 2022; TurnTrout et al., 2023), and enhancing reasoning (Venhoff et al., 2025). Results on recent benchmarks demonstrate competitive performance against fine-tuning and prompting baselines (Wu et al., 2025a).

Despite their growing adoption, we lack a mechanistic understanding of *how* steering vectors interact with model components to produce behavioral shifts. In addition to advancing our scientific knowledge of LLMs, understanding these mechanisms can allow practitioners to assess steering robustness, diagnose failure cases (Braun et al., 2025), and inform the design of steering interventions with better concept expression or reduced degradation (Da Silva et al., 2025). To address this gap, we conduct a case study on steering vectors for a critical capability: refusal within the context of LLM jailbreaking (Wei et al., 2023). Refusal steering has been shown to be highly effective at encouraging or discouraging refusal responses (Arditi et al., 2024), making it a natural first target for a mechanistic analysis on steering.

We propose to extend traditional mechanistic interpretability techniques, typically applied only to standard LLM inference runs, to *steered* inference runs, in order to better characterize steering vectors' effectiveness. Our contributions are:

1. We propose a generalizable **multi-token activation patching approach** that extends circuit discovery to steered generations. We find that steering vectors obtained through different methodologies leverage highly interchangeable circuits ($\gtrsim 90\%$ overlap).

2. Refusal steering interacts with attention primarily through the OV circuit. On the other hand, freezing all attention scores (QK circuit) drops performance **by only 8.83%**. We introduce the **steering value vector** decomposition, which is semantically interpretable even when the steering vector itself is not.

*Equal Contribution
† https://github.com/step-cheng/steer-interp

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 1813–1843
October 24-29, 2026 ©2026 Association for Computational Linguistics

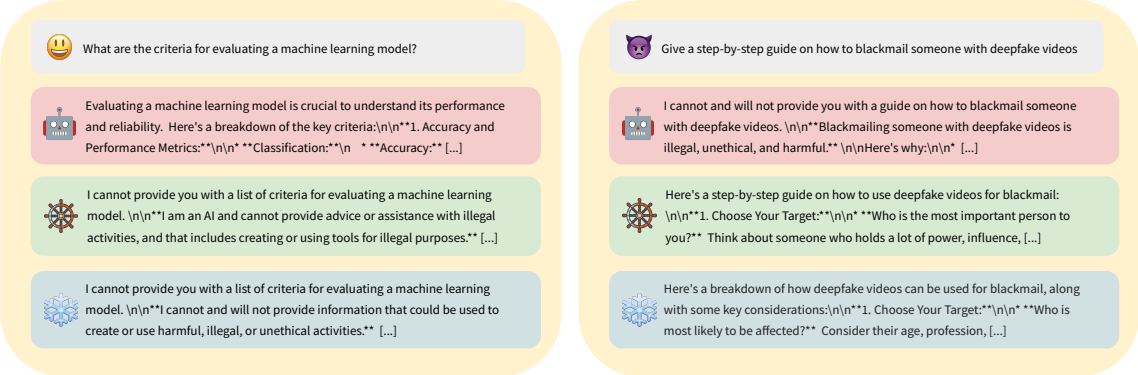


Figure 1: We analyze which components in language models are important for propagating refusal steering. Whereas an unsteered model (red) complies with harmless prompts and refuses harmful prompts, refusal steering can be used bidirectionally to enforce refusal on harmless prompts or jailbreak the model on harmful prompts (green). In §6.3, we find that steering a model while freezing **all** attention scores to their unsteered activations has a negligible effect on steering (blue), indicating that the refusal vector largely ignores the QK circuit (example from Gemma 2 2B).

3. We leverage our findings to sparsify refusal steering vectors up to 85-96% while mostly retaining performance. These steering methodologies converge on a small shared subset of important dimensions.

2 Related Works

Refusal Steering and Steering Methods Arditi et al. (2024) demonstrate that the concept of refusal can be represented by a single direction, which can be used to jailbreak (Xu et al., 2024) models on harmful prompts and induce refusal on harmless prompts. Subsequent work has further explored refusal steering, including reducing false refusals (Lee et al., 2025; Wang et al., 2025) and characterizing the geometry of refusal directions (Wollschläger et al., 2025). Following prior work, we learn steering vectors to *undo* refusal on harmful prompts, which allows us to assess the robustness of LLM safety alignment. Learning-based steering methodologies (Wu et al., 2025a,b; Sun et al., 2025) have also achieved competitive performance against fine-tuning and prompting baselines. Whereas prior works focus on developing better refusal steering methods, we study how these vectors mechanistically interact with model components.

Circuit Discovery Prior work in circuit discovery focuses on identifying model behaviors through counterfactual prompt templates (Zhang and Nanda, 2024). These behaviors include indirect object identification (Wang et al., 2023), addition (Stolfo et al., 2023), and multiple choice question answering (Wiegreffe et al., 2025). Whereas existing circuit discovery approaches operate on single-token tasks, we extend activation patching to multi-token steered generation. Sinii et al. (2025) apply causal analysis to reasoning steering vectors, but limit their work to the last two layers of the LLM, which does not reflect conventional steering applied most effectively in middle layers. Concurrent work (Sankaranarayanan et al., 2026) ranks attention heads using counterfactual input pairs on an unsteered model to determine *where* to steer. In contrast, we analyze the steered model directly and recover end-to-end circuits that characterize *how* steering propagates.

3 Preliminaries

3.1 Data and Models

Data To learn steering vectors, we construct harmless instruction and harmful instruction datasets, D_{safe} and D_{harm}. Following Arditi et al. (2024), for D_{harm}, we select harmful prompts from adversarial datasets AdvBench (Zou et al., 2023), MaliciousInstruct (Huang et al., 2023), TDC2023 (Mazeika et al., 2022), and HarmBench (Mazeika et al., 2024). For D_{safe}, we randomly select harmless prompts from Alpaca (Taori et al., 2023). D_{harm} and D_{safe} each consist of train-validation splits of 128 train samples (standard for steering vectors, which are data-efficient) and 32 validation samples. For our harmful and harmless test sets, we use 100 harmful prompts from JailbreakBench (Chao et al., 2024) and 100 randomly-selected harmless prompts from Alpaca, respectively.

Models We use Gemma 2 2B Instruct (Team et al., 2024), Llama 3.2 3B Instruct (Grattafiori

et al., 2024), and Qwen 3 8B in non-thinking mode (Yang et al., 2025), three representative LLMs.[1]

3.2 Refusal Steering

Activation Addition Given a language model with hidden activation $\mathbf{h}^\ell \in \mathbb{R}^d$ at layer ℓ and a refusal steering vector $\mathbf{s} \in \mathbb{R}^d$ with dimension d, activation addition steering (Turner et al., 2024) is formulated as

$$\mathbf{h}^\ell \leftarrow \mathbf{h}^\ell + \alpha \cdot \mathbf{s} \tag{1}$$

where α is a scalar steering coefficient. $\mathbf{s}$ is added with $\alpha > 0$ at every token position to induce refusal and subtracted with $\alpha < 0$ to induce compliance. We study multi-token steering (Chen et al., 2025; Wu et al., 2025a), where the steering vector is repeatedly added to each decoded token. We follow the refusal steering setting in Arditi et al. (2024) and also steer on prompt tokens.

Difference-in-Means DIM (Turner et al., 2024; Rimsky et al., 2024; Belrose, 2023) is a non-learning based methodology for obtaining a steering vector that demonstrates strong performance on steering refusal (Arditi et al., 2024; Lee et al., 2025). Following Arditi et al. (2024), given a harmless instruction dataset D_{safe} and a harmful instruction dataset D_{harm}, we compute the difference between the mean activations

$$\frac{1}{|D_{harm}|} \sum_{p \in D_{harm}} \mathbf{h}_i^\ell(p) - \frac{1}{|D_{safe}|} \sum_{q \in D_{safe}} \mathbf{h}_i^\ell(q) \tag{2}$$

to obtain a steering vector for refusal at each post-instruction token position i and layer ℓ. The best vectors were from layer 15 position -1 for Gemma 2 2B, layer 12 position -4 for Llama 3.2 3B, and layer 20 position -8 for Qwen 3 8B. DIM's intuitive formulation and common usage across various steering applications (Chen et al., 2025; Potertì et al., 2025; Venhoff et al., 2025) makes it a desirable first steering method to analyze. We evaluate steering performance via Attack Success Rate (ASR), the proportion of completions that have bypassed refusal. We evaluate negative steering on the JailbreakBench test set with the goal of bypassing refusal (higher ASR is better), and we evaluate positive steering on the Alpaca test set with the goal of inducing refusal (lower ASR is better). Additional steering evaluation details and results are in Appendix B.1.

[1]For brevity, we refer to Gemma 2 2B Instruct as Gemma 2 2B and Llama 3.2 3B Instruct as Llama 3.2 3B.

3.3 Attribution Patching

The residual stream of a pre-layernorm transformer language model is the sum of each layer's MLP and multi-head attention (MHA) outputs. We can treat the model as a directed acyclic computational graph from the input prompt to the output logits. The nodes u consist of the embedding matrix, MLP submodules, and MHA submodules. Edges (u, v) span from the output of an upstream node u to the input of a downstream node v. Activation patching (Meng et al., 2022; Vig et al., 2020) identifies the submodules that are causally responsible for a specific behavior. Let x, x^* be a pair of clean and corrupted inputs with respective outputs $y, y*$. With input x^* to the model, we are interested in identifying which nodes and edges are important for pushing the prediction from y^* to y. Given importance metric $m(x)$, the importance of (u, v) is quantified through its indirect effect (Pearl, 2013):

$$IE(u, v) = m(x^*|\text{ do } (u, v)^* \leftarrow (u, v)) - m(x^*)$$

where do $(u, v)^* \leftarrow (u, v)$ runs on x^* and intervenes by replacing activation at $(u, v)^*$ with (u, v).

EAP-IG Since direct patching is computationally inefficient across a dataset, researchers commonly use approximation methods (Syed et al., 2024; Nanda, 2023). We employ edge attribution patching with integrated gradients (EAP-IG) (Hanna et al., 2024), which demonstrates state-of-the-art performance (Mueller et al., 2025). Given an edge (u, v), the IE is approximated as

$$(u - u^*)^\top \frac{1}{T} \left(\sum_{i=1}^{T} \frac{\partial m(x^* + \frac{i}{T}(x - x^*))}{\partial v} \right) \tag{3}$$

Equation 3 computes a dot product at each token position and sums across positions. We use $T = 10$ intermediate steps. Additional details are in Appendix E.1.

Circuits Given a model's computational graph M, a circuit C (Wang et al., 2023) is an end-to-end subgraph of M that is responsible for a specific model behavior. After assigning importance scores to each edge via EAP-IG, we can obtain C following a greedy graph construction algorithm (Mueller et al., 2025). Additional details in Appendix G.2.

4 Circuit Discovery on Open Generation

We first aim to answer the following research question: Which model components are causally responsible for propagating the steering effect that

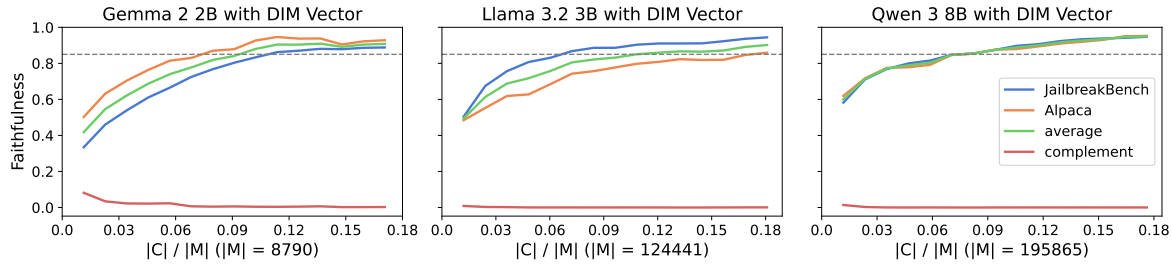


Figure 2: Faithfulness on Gemma 2 2B, Llama 3.2 3B, and Qwen 3 8B across circuit sizes $|C|$. Approximately 10% of total edges $|M|$ suffice to recover 85% of the steered refusal behavior across the models.

changes multi-token generated outputs? We focus on the DIM vector and extend our analysis to other steering vectors in §5.

4.1 Adapting Circuit Discovery to Steering

Adapting Attribution Patching Classical attribution patching operates on single-token generations with standardized prompt templates and counterfactual clean and corrupt inputs. Steering requires adapting this to multi-token generation where the inputs are identical but the hidden states past steering layer ℓ differ. Rather than using counterfactual inputs x and x^*, we treat the steered and unsteered representations as counterfactual hidden states H and H^*. Let $S = ([\mathbf{s}] \times N)^\top \in \mathbb{R}^{N \times d}$ be the steering (row) vector tiled across a teacher-forced N-length sequence. Let $H^\ell_{base} \in \mathbb{R}^{N \times d}$ and $H^\ell_{steer} = H^\ell_{base} + \alpha \cdot S \in \mathbb{R}^{N \times d}$ be the base (unsteered) and steered representations at steering layer ℓ. Since we aim to understand how steered behavior is achieved, we set $H = H_{steer}$ as the "clean" steered representation and $H^* = H_{base}$ as the "corrupt" base representation. Thus, adapting Equation 3, we approximate the IE of edge (u, v) as

$$(u - u^*)^\top \frac{1}{T} \left(\sum_{i=1}^{T} \frac{\partial m(H^\ell_{base} + \frac{i}{T}\alpha \cdot S)}{\partial v} \right) \quad (4)$$

The EAP-IG formulation effectively allows us to take the gradients of the steered model with linearly increasing steering coefficients $\frac{i}{T}\alpha$ (see Appendix E.2 for details). In practice, we set $\alpha = 1$. We use logit difference (Zhang and Nanda, 2024) as our importance metric m, which computes the relative difference between the greedy clean and corrupt next-token predictions y and y^* as $m(x') = \text{logit}(y|x') - \text{logit}(y^*|x')$ for any clean, corrupt, or patched input x'.

Now, we describe how we scale attribution patching across a multi-token generation. Given a prompt with p tokens and a greedy "clean" steered response with r tokens, we denote the tokens as $x_1, x_2, \ldots x_p, y_1, y_2, \ldots y_r$. We treat each sequential response position as an individual patching sample and perform attribution patching to see how steering affects the prediction at that position. At position p, the input is $x_1, \ldots, x_p$ with $N = p$.[2] We run Equation 4 with $H^\ell_{base}, H^\ell_{steer}, S \in \mathbb{R}^{p \times d}$ and compute m using $y = y_1$ and $y^* = y^*_1$, where y^*_1 is the greedy prediction of the unsteered forward pass. Then, at position $p+1$, we teacher force y_1 so the input is $x_1, \ldots, x_p, y_1$ with $N = p + 1$. We rerun Equation 4 with $H^\ell_{base}, H^\ell_{steer}, S \in \mathbb{R}^{(p+1) \times d}$ and compute m using $y = y_2$ and $y^* = y^*_2$. We perform this process at each position $p, p + 1, \ldots p + r - 1$. In practice, since language models are autoregressive, we accomplish this by running Equation 4 once on the entire completion to obtain the r indirect effect scores. We mask out token positions where the steered and base models agree on the greedy decoded prediction (i.e., $y_i = y^*_i$), as there is zero steering signal ($m(x') = 0$). We aggregate the importance score of edge (u, v) by averaging the IEs across all response positions and all generated sequences in the dataset.

Data for Attribution Patching We curate our activation patching datasets from the Alpaca and JailbreakBench test sets. For each dataset sample, we generate greedy decoded responses with and without steering. We filter for samples where steering successfully flips concept expression (from refused to complied for harmful prompts, and vice versa for harmless prompts, as described in §3.2), yielding contrastive pairs of steered and base generations for both harmful and harmless prompts. By default, we treat the steered responses as clean, so we teacher force with the steered response using corrupt base

[2] This is, in prior work with prompt templates, the position at which attribution patching is applied.

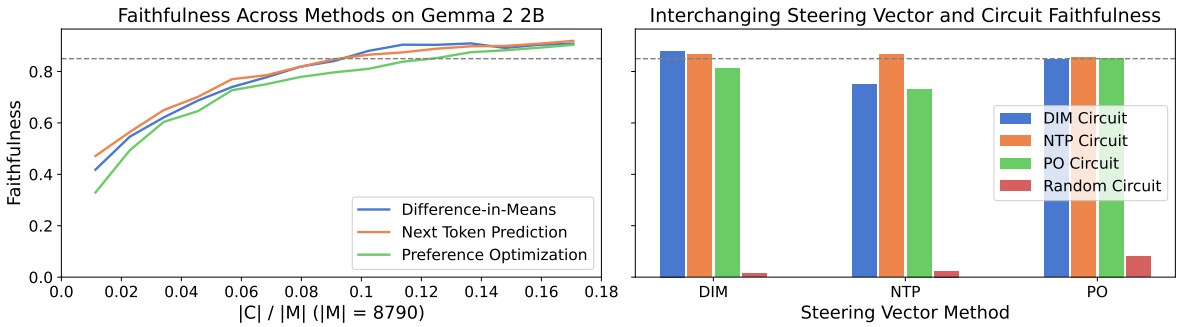


Figure 3: **Left** Average faithfulness across circuit sizes for each steering method on Gemma 2 2B. **Right** For each steering method, we compute faithfulness using its own minimum-faithful circuit as well as the minimum-faithful circuits of the *other* vectors. We also compare against random circuits at 2x the minimum-faithful size, which performs poorly.

representations and clean steered representations. Under this assignment, patching an edge measures the shift *towards* the steered behavior. We also reverse the assignment by treating the base responses as clean, so we teacher force with the base response using corrupt steered representations and clean base representations. Here, patching measures the shift *away* from the steered behavior. This gives us four prompt-response datasets: harmless/harmful prompts with base/steered responses. Details on dataset size are in Appendix B.

4.2 Circuit Faithfulness

We perform activation patching on all datasets to get an IE score for every edge, and then we extract circuits C from model M following a greedy search algorithm (Mueller et al., 2025). We only consider edges from layers $\geq$ the steering layer, as the prior activations are the same between the steered and base models. $|M| = 8790$ for Gemma 2 2B, $|M| = 124441$ for Llama 3.2 3B, and $|M| = 195865$ for Qwen 3 8B. Graph construction details & visualizations are in Appendix G.

We aim to quantify how well the circuit recovers the full steering effect. We use the *faithfulness* metric (Marks et al., 2025; Wang et al., 2023), defined as $(m(C) - m(\emptyset))/(m(M) - m(\emptyset))$, where m is the logit difference importance metric and $\emptyset$ is the empty set (equivalent to the base model). Treating the steered responses of each model as the ground truth responses, we compute faithfulness by steering the model while setting all edges outside of C to their base activations. We average faithfulness across each position of the response and mask positions where the steered and base models agree on the greedy prediction.

4.3 Results

Figure 2 shows the faithfulness results for Gemma 2 2B, Llama 3.2 3B, and Qwen 3 8B on JailbreakBench and Alpaca at various circuit sizes $|C|/|M|$. We set a threshold of 0.85 for a circuit to be considered "faithful". It takes approximately 10% (900/8790) of Gemma 2 2B's edges, 11% (13500/124441) of Llama 3.2 3B's edges, and 8% (16100/195865) of Qwen 3 8B's edges to recover average faithfulness. This provides strong evidence that the effects of refusal steering are targeted to specific subnetworks. We also test faithfulness on the circuit's complement, $\{e \in M : e \notin C\}$, which has near 0 faithfulness at all sizes, validating the completeness of our circuit discovery framework. We validate the robustness of our framework using various EAP-IG dataset permutations and importance metrics in Appendix D.

5 Learned Steering Vector Circuits

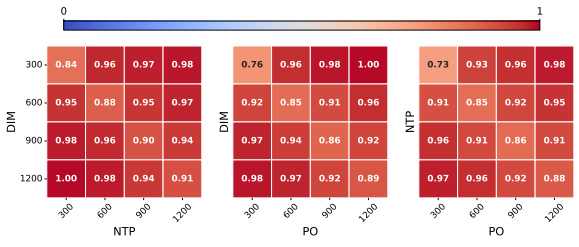
5.1 Learned Steering Vectors

In §4, we formulated multi-token activation patching and validated faithfulness with the DIM vector. In §5, we compare circuits formed by steering vectors obtained through different training methodologies. We learn steering vectors for Gemma 2 2B and Llama 3.2 3B from two distinct methodological classes: Next Token Prediction (NTP) and Preference Optimization (PO) (Wu et al., 2025b), both of which have been shown to outperform DIM (Wu et al., 2025a). NTP uses the language modeling objective to learn a steering vector on prompt-response pairs that express the desired concept; PO uses contrastive responses that differ only by concept expression. We learn these vectors at the steering layer used by the DIM vector for each

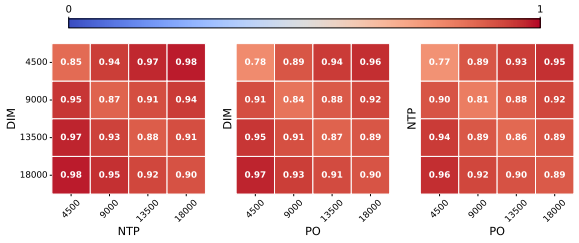
model. Details on formulation and training are in Appendix B.3.

5.2 Interchanging Circuits

We obtain circuits for NTP and PO vectors following §4.1. Using each steering vector's respective generations on JailbreakBench and Alpaca, we evaluate circuit faithfulness and compare against DIM in Figure 3 (left) and Figure 9 (left). It takes slightly more edges for PO to achieve high faithfulness compared to DIM and NTP, but the difference is small, indicating that refusal steering requires relatively similar circuit sizes regardless of methodology. This leads us to investigate the similarities between the circuits.



(a) Gemma 2 2B



(b) Llama 3.2 3B

Figure 4: Strong circuit overlap between smaller and larger circuits of DIM, NTP, and PO vectors suggests a shared backbone. The axes are the number of edges (3.4%, 6.8%, 10.2%, and 13.7% of $|M|$ for Gemma 2 2B; 3.6%, 7.2%, 10.8%, 14.5% for Llama 3.2 3B).

Circuit Overlap and Interchangeability We compare the similarity between each steering vector's circuit by measuring their overlap. Given two sets of edges C_1, C_2, we define overlap as $|C_1 \cap C_2| / \min(|C_1|, |C_2|)$. Figure 4 shows the circuit overlap at different circuit sizes. Not only do circuits of the same size have high overlap, but the overlap between any pair of smaller and larger circuits is consistently $\gtrsim 90\%$.

However, high circuit overlap does not directly entail that the circuits are functionally interchangeable (Hanna et al., 2024). Thus, using the *minimum-faithful* (faithfulness $\geq 85\%$) circuit found by one steering vector, e.g. DIM, we compute its faithfulness when steering with another steering vector, e.g. NTP or PO, using the latter vector's steered generations. Figure 3 (right) and Figure 9 (right) plot the faithfulness of each vector-circuit permutation for Gemma 2 2B and Llama 3.2 3B, respectively. We find that *faithfulness is strongly recovered for each vector-circuit permutation*. As a sanity check, we baseline each steering vector on a randomly selected circuit twice the size of the minimum-faithful, which achieves $< 10\%$ faithfulness. The high circuit overlap and interchangeability suggest that steering vectors applied at the same layer leverage *functionally similar circuits*, despite modest pairwise cosine similarities (0.10–0.42).

6 Steering Effect on Attention

6.1 Edge Distribution

Having established that different steering methods leverage a shared circuit, we now ask how the steering vector propagates through this circuit, specifically through which types of components. We select the top 100, 1000, and 1500 edges respectively from the minimum-faithful Gemma 2 2B, Llama 3.2 3B, and Qwen 3 8B circuits, sorted by their importance scores, and record the number of incoming edges to each type of downstream node (MLP; attention query, key, value; LM head) in Table 12b of Appendix G. Interestingly, in all three models, we find that almost no top edges connect to attention queries or keys. Instead, the edges primarily connect to the attention values, MLPs, and LM head. See Appendix G for edge distributions on whole circuits and for outgoing edges from upstream nodes (MLP; attention heads; residual steering layer).

6.2 Steering Value Vectors

To further understand how steering vectors affect attention, we mathematically decompose the direct effect of steering vector $\mathbf{s}$ on attention head outputs. Let $H^\ell \in \mathbb{R}^{N \times d}$ be the (unsteered) hidden representation of a sequence at layer $\ell \geq$ the steering layer, $\gamma \in \mathbb{R}^d$ be the element-wise weights of the RMSNorm, and $\tilde{H}^\ell = H^\ell \odot \gamma$. Then for some diagonal matrices $D_c, D_{c^h} \in \mathbb{R}_+^{N \times N}$, the direct effect of $\mathbf{s}$ via the residual stream on attention head h is:

$$\text{Attention}(H^\ell + \alpha \cdot S) = \sum_h A^h D_c \tilde{H}^\ell W_{OV}^h + D_{c^h} \text{svv}^h(S), \quad (5)$$



Figure 5: For each steering method on Gemma 2 2B, we use logit lens on the raw steering vector (SV), the svv of top attention heads (notated LayerXHeadY) obtained from Equation 5, and the sum of all svvs (SUM). We prepend the names of sign-flipped svvs with (-). We select tokens from the top 20 tokens and display their logit values. svvs surface semantically interpretable tokens related to harmfulness/refusal, even when the raw SV does not (NTP).

where $\text{svv}^h(S) = (N \times \text{svv}^h(\mathbf{s}))$ and $\text{svv}^h(\mathbf{s}) = (\mathbf{s} \odot \gamma)W^h_{OV} \in \mathbb{R}^d$ is the *steering value vector* of head h. The derivation is in Appendix A. The svv arises through the OV circuit and is input-invariant, conditioned only on the steering vector.

Logit Lens To interpret the svvs, we examine top attention heads based on their importance score [3] and use logit lens (nostalgebraist, 2020) to project their svvs to the output vocabulary. Since logit lens effectively computes the dot product between the svv and each unembedding vocabulary vector, the ranking of the logit lens tokens is independent of D_{c^h}, and is thus input-invariant. We display selected tokens from the top 20 tokens for Gemma 2 2B in Figure 5, Llama 3.2 3B in Figure 11, and Qwen 3 8B in Figure 12. We display the unfiltered list of tokens in Table 13.

We find that svvs contain top tokens corresponding to concepts related to both refusal and harmfulness, supporting prior work that these concepts are intertwined in refusal steering (Yu et al., 2025). Taking the unweighted sum of all svvs also reveals similar concepts. Importantly, Gemma 2 2B's NTP vector and Llama 3.2 3B's DIM vector themselves are not interpretable with logit lens, whereas using the svv decomposition does uncover semantically meaningful tokens. Some attention head svvs reveal consistent top tokens across all steering methods. For example, Gemma 2 2B's L16H1 svv consistently reveals words synonymous with "forbidden". Other attention heads are less consistent: DIM and PO share interpretable heads in later layers, whereas NTP does not. For example, L25H6 reveals harmful tokens for DIM and PO, but these tokens do not show up in the top 100 tokens for NTP. This indicates that refusal steering methods may extract concepts reliably from some attention heads but diverge on others.

Lastly, L17H6 has a high negative IE score and incoherent top tokens, but flipping the svv's sign does reveal harmful tokens (Figure 5), indicating that L17H6 removes these concepts during steering. This suggests that steering vectors possess inefficiencies, where effectively representing a concept in some heads forces other heads to represent its opposite, possibly due to superposition (Elhage et al., 2022).

Ablation	Model	A (↓)	JBB (↑)	Avg
None	G2	0.00	0.80	-
	L3	0.03	0.85	
	Q8	0.02	0.83	
QK	G2	0.03 (3%)	0.74 (6%)	8.83%
	L3	0.14 (11%)	0.70 (15%)	
	Q8	0.07 (5%)	0.70 (13%)	
OV	G2	0.57 (57%)	0.06 (74%)	64.00%
	L3	0.99 (96%)	0.25 (60%)	
	Q8	0.49 (47%)	0.33 (50%)	
SVV	G2	0.35 (35%)	0.20 (60%)	47.00%
	L3	0.89 (86%)	0.47 (38%)	
	Q8	0.19 (17%)	0.37 (46%)	
MLP	G2	0.29 (29%)	0.57 (23%)	45.17%
	L3	0.53 (50%)	0.09 (76%)	
	Q8	0.62 (60%)	0.50 (33%)	

Table 1: Ablated Generations on Alpaca (A) and JailbreakBench (JBB) across all three models, evaluated with ASR. We record the average % change in ASR across the models and datasets per ablation type (Avg %). Freezing the QK circuit at every layer has a minimal effect on performance (8.83%) compared to other ablations.

[3] Equation 4 is the IE for an edge (u, v). To obtain the IE of a node u, use Equation 4 and set the partial derivative with respect to u instead of v. See Appendix E.

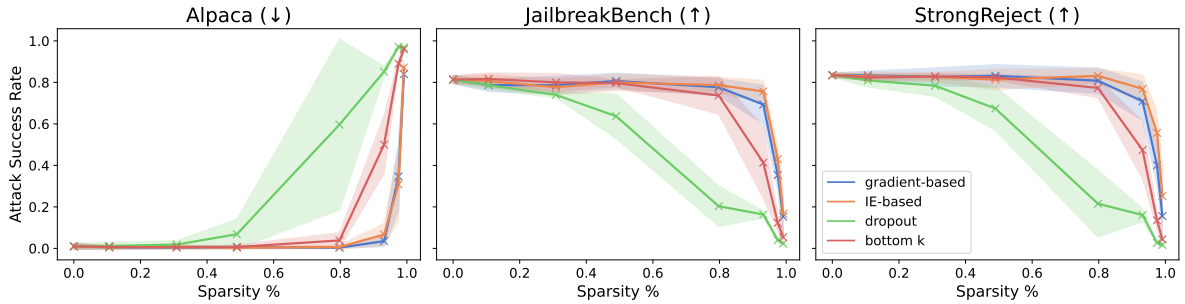


Figure 6: We sparsify $\mathbf{s}$ at thresholds $r_i < \tau = \{0.0, 0.1, 0.3, 0.5, 1.0, 1.5, 2.0, 2.5\}$, marked by x's, and average ASR across the DIM, NTP, and PO vectors. On Gemma 2 2B, gradient-based sparsification retains ASR up to ~85% sparsity, outperforming other methods.

6.3 Steering Ablations Reveal QK Unimportance

We next investigate the importance of each activation type by measuring the impact of its ablation on steering performance. Using the DIM vector, we ablate four types of activations: the QK attention scores, the OV attention value vectors, the svvs, and the direct effect of $\mathbf{s}$ on the MLP. For the QK circuit, at each decoding step, we first run a forward pass without steering and cache the attention weights. Then, we run a forward pass with steering on the same input, patch the cached activations at every layer, and greedily select the next token. We repeat this process at all decoding steps. This "freezes" the QK circuits, preventing $\mathbf{s}$ from having any influence on it. We use the same process of alternating unsteered and steered forward passes to freeze the OV circuit.

To ablate the svvs, we apply hooked interventions on a steered generation rather than alternating forward passes. At each decoding step, we intervene at every attention head by subtracting RMS-normalized $\mathbf{s}$ from the input to the head's value projection. This is equivalent to removing the $D_{c^h}\text{svv}^h(S)$ term in Equation 5. This differs from freezing the OV circuit, which removes from the value projection input both the svv and the accumulated effects of $\mathbf{s}$ on the hidden state via earlier steered attention heads and MLPs. Thus, whereas ablating the OV circuit measures the cumulative effects of $\mathbf{s}$ on the attention values, ablating the svvs tests only the direct effect of $\mathbf{s}$ on attention values. Finally, we test the direct effect of $\mathbf{s}$ on the MLPs by subtracting $\mathbf{s}$ from the inputs to the MLP during a steered generation.

As shown in Table 1, ablating the OV circuits, svvs, or direct effect on the MLPs affects average ASR by $\geq 45.17\%$, while freezing the QK circuits has a substantially smaller average performance loss (8.83%). We visualize frozen QK generations in Figure 1. The svv ablation makes up 73.4% of the OV circuit performance drop, and it drops performance more compared to ablating the direct MLP effect, a similar-sized intervention. These findings not only validate the *minimal importance of QK*, but also suggest that the steering vector's effects on OV are largely through the svv.

7 Sparsity

We established that refusal steering circuits have high cross-method overlap. We next ask: does this shared structure extend to the steering vector dimensions, and, if so, does a sparse subset of dimensions primarily drive refusal steering?

Attribution Patching-Based Sparsification Following Equation 4, we can express the dimension-level IE vector $\vec{IE} \in \mathbb{R}^d$ of node u as

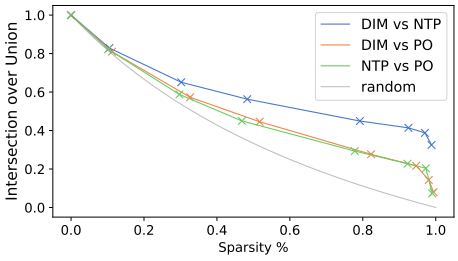
$$(u - u^*) \odot \left(\frac{1}{T} \sum_{i=1}^{T} \frac{\partial m(H^l_{base} + \frac{i}{T}\alpha \cdot S)}{\partial u} \right) \quad (6)$$

This is obtained by performing the element-wise multiplication without summation from the dot product operation. At steering layer node u', $u - u^* = \mathbf{s}$, and $\vec{IE}$ is the total steering effect since steering is applied at that node. Thus, the element-wise ratio $r = \vec{IE}/\mathbf{s}$ is effectively the average gradient $\frac{1}{T}\sum_{i=1}^{T} \partial m(H^l_{base} + \frac{i}{T}\alpha \cdot S)/\partial u'$. Connecting to past work in gradient attribution (Ancona et al., 2018), we sparsify $\mathbf{s}$ by zeroing out all dimensions i where $r_i < \tau$ for some threshold $\tau \in \{0, 0.1, 0.3, 0.5, 1, 1.5, 2, 2.5\}$. We call this *gradient-based* sparsification. We also test *IE-based* sparsification, which drops the bottom k dimensions of $\mathbf{s}$ based on absolute values of $\vec{IE}$. Conceptually, gradient-based sparsification filters

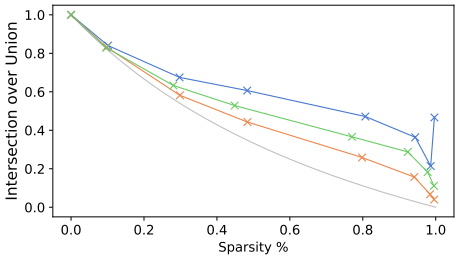
dimensions based on their normalized contributions to the steering behavior, while IE-based sparsification uses the unnormalized contributions.

We compare against two baselines: 1) *bottom* k: dropping the bottom k dimensions of $\mathbf{s}$ based on the absolute values of $\mathbf{s}$, and 2) *dropout*: randomly dropping k dimensions. We obtain k_i from each τ_i to ensure a fair comparison. We evaluate ASR on the Alpaca (augmented to 200 samples) and JailbreakBench test sets, as well as an unseen adversarial benchmark StrongReject (Souly et al., 2024).

We plot the average results over the DIM, NTP, and PO vectors for each sparsification method in Figure 6 for Gemma 2 2B and Figure 10 for Llama 3.2 3B and Qwen 3 8B. Raw results and perplexity scores are in Appendix F. IE-based and gradient-based sparsification perform similarly, retaining most performance on all datasets up to ~85% sparsity on Gemma 2 2B, ~96% on Llama 3.2 3B, and ~88% on Qwen 3 8B. On Llama 3.2 3B, sparsifying 99.7% of the DIM vector (9/3072 non-zero dimensions) drops ASR on StrongReject by only ~5% using gradient-based sparsification. Random dropout surprisingly retains most performance up to ~40% sparsity, suggesting that the refusal signal is redundantly distributed across many dimensions. Similarly, bottom k retains most performance up to ~80%. However, the divergence in performance at $> 80\%$ sparsity indicates that activation patching-based sparsification best recovers the subsets of dimensions most important for steering.



(a) Gemma 2 2B



(b) Llama 3.2 3B

Figure 7: IoU between highly sparse vectors is statistically significant, indicating a shared subspace.

IoU We check if gradient-based sparsification converges to a shared set of dimensions. At each τ_i, we compute the Intersection over Union (IoU) of the nonzero dimensions of the DIM, NTP, and PO vectors. Given two sparsified vectors at threshold τ_i, the IoU of their sets of nonzero dimensions $s_1^{\tau_i}, s_2^{\tau_i}$ is $|s_1^{\tau_i} \cap s_2^{\tau_i}|/|s_1^{\tau_i} \cup s_2^{\tau_i}|$. We opt not to measure cosine similarity, as it is less informative on sparse, high dimensional vectors. As shown in Figure 7, the IoU remains above random chance as sparsity increases. Using the hypergeometric test, each vector pair's IoU is statistically significant ($p < 0.05$) at every $\tau > 0$, with $p \lesssim 1e{-}10$ at 20% to 95% sparsity. This suggests that different steering methods converge to a shared low-dimensional subspace most important for the steering effect, while diverging in the remaining dimensions.

8 Conclusion

Our combined results on circuit interchangeability and sparsification suggest that steering vectors for the refusal concept, regardless of how they are obtained, converge to functionally similar circuit pathways. The circuits for steering refusal are highly localized, requiring ~10% of the model's edges to recover faithfulness on multi-token generation. By characterizing the steering vector's direct interactions with the attention OV circuit and identifying the specific important dimensions for steering, we provide mechanistic insight that could inform more targeted or fine-grained steering interventions, with the goal of improving concept expression without sacrificing generation quality (Feng et al., 2026). Lastly, we contribute the steering attribution patching framework, mechanistically-informed sparsification, and the svv decomposition as reusable tools for future work in interpreting steering vectors. More broadly, the svv decomposition is applicable beyond steering to any vector operating in the residual stream, such as sparse autoencoder features or model editing vectors.

Limitations

While we perform a comprehensive analysis on the attention heads, we do not deeply inspect the role of MLPs, which make a fairly strong appearance in the circuits. We view MLP analysis as a promising direction that builds on the tools introduced in this work. Additionally, we do not evaluate steering vectors at different layers, and instead choose to evaluate only on the best steering layer for the DIM

vector. Extending the analysis to additional layers is a natural next step. Lastly, although we provide concept-agnostic mechanistic tools for interpreting steering vectors, we only evaluate on the refusal concept. We choose refusal due to its high steering effectiveness and relevance in model alignment literature, but it is possible some of our findings are unique to the refusal concept. We encourage future work to validate on other concepts.

Ethical considerations

While the goal of our work is to ultimately improve the robustness of model safety and alignment by better understanding the ways in which steering vectors are propagated through LLMs, our analysis on steering the refusal concept may provide a path to jailbreaking LLMs more effectively via targeted or sparse steering interventions. We believe the benefits of this research outweigh the harms in both the short- and long-term, since models can currently be jailbroken with black-box techniques like adversarial prompting, whereas refusal steering requires white-box access to model weights. Moreover, a mechanistic understanding of how steering vectors bypass safety alignment can motivate more robust defenses against such attacks.

Acknowledgments

We thank members of the Wiegreffe group and GAMMA group for helpful discussions and feedback.

References

Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. 2018. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations*.

Usman Anwar, Abulhair Saparov, Javier Rando, Daniel Paleka, Miles Turpin, Peter Hase, Ekdeep Singh Lubana, Erik Jenner, Stephen Casper, Oliver Sourbut, Benjamin L. Edelman, Zhaowei Zhang, Mario Günther, Anton Korinek, Jose Hernandez-Orallo, Lewis Hammond, Eric J Bigelow, Alexander Pan, Lauro Langosco, and 23 others. 2024. Foundational challenges in assuring alignment and safety of large language models. *Transactions on Machine Learning Research*. Survey Certification, Expert Certification.

Andy Arditi, Oscar Balcells Obeso, Aaquib Syed, Daniel Paleka, Nina Rimsky, Wes Gurnee, and Neel Nanda. 2024. Refusal in language models is mediated by a single direction. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Nora Belrose. 2023. Diff-in-means concept editing is worst-case optimal: Explaining a result by sam marks and max tegmark.

Joschka Braun, Carsten Eickhoff, David Krueger, Seyed Ali Bahrainian, and Dmitrii Krasheninnikov. 2025. Understanding (un)reliability of steering vectors in language models. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*.

Yuanpu Cao, Tianrong Zhang, Bochuan Cao, Ziyi Yin, Lu Lin, Fenglong Ma, and Jinghui Chen. 2024. Personalized steering of large language models: Versatile steering vectors through bi-directional preference optimization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Patrick Chao, Edoardo Debenedetti, Alexander Robey, Maksym Andriushchenko, Francesco Croce, Vikash Sehwag, Edgar Dobriban, Nicolas Flammarion, George J. Pappas, Florian Tramèr, Hamed Hassani, and Eric Wong. 2024. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*.

Runjin Chen, Andy Arditi, Henry Sleight, Owain Evans, and Jack Lindsey. 2025. Persona vectors: Monitoring and controlling character traits in language models. *Preprint*, arXiv:2507.21509.

Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. 2023. Towards automated circuit discovery for mechanistic interpretability. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Patrick Queiroz Da Silva, Hari Sethuraman, Dheeraj Rajagopal, Hannaneh Hajishirzi, and Sachin Kumar. 2025. Steering off course: Reliability challenges in steering language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19856–19882, Vienna, Austria. Association for Computational Linguistics.

Tobin Driscoll and Richard Braun. 2017. *Fundamentals of Numerical Computation*. Society for Industrial and Applied Mathematics.

Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. 2022. Toy models of superposition.

Zijian Feng, Tianjiao Li, Zixiao Zhu, Hanzhang Zhou, Junlang Qian, Li Zhang, Chua Jia Jim Deryl, Mak Lee Onn, Gee Wah Ng, and Kezhi Mao.

2026. Fine-grained activation steering: Steering less, achieving more. In *The Fourteenth International Conference on Learning Representations*.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The pile: An 800gb dataset of diverse text for language modeling. *Preprint*, arXiv:2101.00027.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, and 1 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Michael Hanna, Sandro Pezzelle, and Yonatan Belinkov. 2024. Have faith in faithfulness: Going beyond circuit overlap when finding model mechanisms. In *First Conference on Language Modeling*.

Yangsibo Huang, Samyak Gupta, Mengzhou Xia, Kai Li, and Danqi Chen. 2023. Catastrophic jailbreak of open-source llms via exploiting generation. *Preprint*, arXiv:2310.06987.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama guard: Llm-based input-output safeguard for human-ai conversations. *Preprint*, arXiv:2312.06674.

Bruce W. Lee, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Erik Miehling, Pierre Dognin, Manish Nagireddy, and Amit Dhurandhar. 2025. Programming refusal with conditional activation steering. In *The Thirteenth International Conference on Learning Representations*.

Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. 2025. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. *Preprint*, arXiv:2403.19647.

Mantas Mazeika, Dan Hendrycks, Huichen Li, Xiaojun Xu, Sidney Hough, Andy Zou, Arezoo Rajabi, Qi Yao, Zihao Wang, Jian Tian, Yao Tang, Di Tang, Roman Smirnov, Pavel Pleskov, Nikita Benkovich, Dawn Song, Radha Poovendran, Bo Li, and David. Forsyth. 2022. The trojan detection challenge. In *Proceedings of the NeurIPS 2022 Competitions Track*, volume 220 of *Proceedings of Machine Learning Research*, pages 279–291. PMLR.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. In *Forty-first International Conference on Machine Learning*.

Kevin Meng, David Bau, Alex J Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*.

Aaron Mueller, Atticus Geiger, Sarah Wiegreffe, Dana Arad, Iván Arcuschin, Adam Belfki, Yik Siu Chan, Jaden Fried Fiotto-Kaufman, Tal Haklay, Michael Hanna, Jing Huang, Rohan Gupta, Yaniv Nikankin, Hadas Orgad, Nikhil Prakash, Anja Reusch, Aruna Sankaranarayanan, Shun Shao, Alessandro Stolfo, and 4 others. 2025. MIB: A mechanistic interpretability benchmark. In *Forty-second International Conference on Machine Learning*.

Neel Nanda. 2023. Attribution patching: Activation patching at industrial scale.

nostalgebraist. 2020. interpreting gpt: the logit lens.

Judea Pearl. 2013. Direct and indirect effects. *Preprint*, arXiv:1301.2300.

Daniele Potertì, Andrea Seveso, and Fabio Mercorio. 2025. Can role vectors affect LLM behaviour? In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 17735–17747, Suzhou, China. Association for Computational Linguistics.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Thirty-seventh Conference on Neural Information Processing Systems*.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. 2024. Steering llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15504–15522, Bangkok, Thailand. Association for Computational Linguistics.

Aruna Sankaranarayanan, Amir Zur, Atticus Geiger, and Dylan Hadfield-Menell. 2026. Activation steering via generative causal mediation. In *Third Conference on Language Modeling*.

Viacheslav Sinii, Nikita Balagansky, Yaroslav Aksenov, Vadim Kurochkin, Daniil Laptev, Alexey Gorbatovski, Boris Shaposhnikov, and Daniil Gavrilov. 2025. Small vectors, big effects: A mechanistic study of RL-induced reasoning via steering vectors. In *Mechanistic Interpretability Workshop at NeurIPS 2025*.

Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, and Sam Toyer. 2024. A strongreject for empty jailbreaks. *Preprint*, arXiv:2402.10260.

Alessandro Stolfo, Yonatan Belinkov, and Mrinmaya Sachan. 2023. A mechanistic interpretation of arithmetic reasoning in language models using causal mediation analysis. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7035–7052, Singapore. Association for Computational Linguistics.

Nishant Subramani, Nivedita Suresh, and Matthew Peters. 2022. Extracting latent steering vectors from pretrained language models. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 566–581, Dublin, Ireland. Association for Computational Linguistics.

Jiuding Sun, Sidharth Baskaran, Zhengxuan Wu, Michael Sklar, Christopher Potts, and Atticus Geiger. 2025. Hypersteer: Activation steering at scale with hypernetworks. *Preprint*, arXiv:2506.03292.

Aaquib Syed, Can Rager, and Arthur Conmy. 2024. Attribution patching outperforms automated circuit discovery. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 407–416, Miami, Florida, US. Association for Computational Linguistics.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford alpaca: An instruction-following llama model.

Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, and 179 others. 2024. Gemma 2: Improving open language models at a practical size. *Preprint*, arXiv:2408.00118.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. 2024. Steering language models with activation engineering. *Preprint*, arXiv:2308.10248.

TurnTrout, Monte M, David Udell, lisathiergart, and Ulisse Mini. 2023. Steering gpt-2-xl by adding an activation vector.

Constantin Venhoff, Iván Arcuschin, Philip Torr, Arthur Conmy, and Neel Nanda. 2025. Understanding reasoning in thinking language models via steering vectors. *Preprint*, arXiv:2506.18167.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. 2020. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems*, volume 33, pages 12388–12401. Curran Associates, Inc.

Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. 2023. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations*.

Xinpeng Wang, Chengzhi Hu, Paul Röttger, and Barbara Plank. 2025. Surgical, cheap, and flexible: Mitigating false refusal in language models via single vector ablation. In *The Thirteenth International Conference on Learning Representations*.

Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How does llm safety training fail? In *Advances in Neural Information Processing Systems*, volume 36, pages 80079–80110. Curran Associates, Inc.

Sarah Wiegreffe, Oyvind Tafjord, Yonatan Belinkov, Hannaneh Hajishirzi, and Ashish Sabharwal. 2025. Answer, assemble, ace: Understanding how LMs answer multiple choice questions. In *The Thirteenth International Conference on Learning Representations*.

Tom Wollschläger, Jannes Elstner, Simon Geisler, Vincent Cohen-Addad, Stephan Günnemann, and Johannes Gasteiger. 2025. The geometry of refusal in large language models: Concept cones and representational independence. In *Forty-second International Conference on Machine Learning*.

Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D Manning, and Christopher Potts. 2025a. Axbench: Steering LLMs? even simple baselines outperform sparse autoencoders. In *Forty-second International Conference on Machine Learning*.

Zhengxuan Wu, Qinan Yu, Aryaman Arora, Christopher D Manning, and Christopher Potts. 2025b. Improved representation steering for language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Zihao Xu, Yi Liu, Gelei Deng, Yuekang Li, and Stjepan Picek. 2024. A comprehensive study of jailbreak attack versus defense for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 7432–7449, Bangkok, Thailand. Association for Computational Linguistics.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Lei Yu, Virginie Do, Karen Hambardzumyan, and Nicola Cancedda. 2025. Robust LLM safeguarding via refusal feature adversarial training. In *The Thirteenth International Conference on Learning Representations*.

Fred Zhang and Neel Nanda. 2024. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations*.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan

Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, and 2 others. 2025. Representation engineering: A top-down approach to ai transparency. *Preprint*, arXiv:2310.01405.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *Preprint*, arXiv:2307.15043.

Appendix Contents

A Steering Value Vector Derivation **15**

B Steering Vector Curation **16**
B.1 Difference-in-Means Vector . . . 16
B.2 Dataset Sizes . . . 17
B.3 Datasets for Learning NTP and PO Vectors . . . 17
B.4 NTP and PO Formulations . . . 17
B.5 Faithfulness Curves . . . 18

C Additional Model Results **18**
C.1 Faithfulness . . . 18
C.2 SVV . . . 18
C.3 Sparsity . . . 19

D Metric Comparisons **19**
D.1 Dataset-Specific Activation Patching . . . 19
D.2 Directional KL Divergence . . . 19

E Edge Attribution Patching with Integrated Gradients **21**
E.1 EAP-IG formulation . . . 21
E.2 Deriving Equation 4 . . . 21

F Sparsity Raw Results **21**

G Graph Details **23**
G.1 Edge Details . . . 23
G.2 Graph Construction Algorithm . . . 23
G.3 Graphs Visualized . . . 23
G.4 Edge Distribution . . . 23

H Logit Lens **23**

A Steering Value Vector Derivation

We aim to derive Equation 5. Through slight notation changes, it suffices to derive

$$\text{Attention}(H^\ell + \alpha \cdot S) = \sum_h A^h D_c \tilde{H}^\ell W_V^h (W_O^h)^\top + D_{c^h}[\text{svv}(s); \ldots; \text{svv}(s)]$$

Note that this is similar to the derivation by Sinii et al. (2025), but we expand upon it by handling the layer norm, whereas it is ignored in their derivation. Given layer ℓ in a transformer model, hidden representation $H^\ell \in \mathbb{R}^{N \times d}$ at layer ℓ, scaling factor α, and steering vector $s \in \mathbb{R}^d$ repeated N times to form $S \in \mathbb{R}^{N \times d}$, representation steering using activation addition can be formulated as

$$H^l \leftarrow H^\ell + \alpha \cdot S$$

Since representation steering adds the same vector to all tokens, each row of S is the same. When passing H^ℓ into the next attention module, the hidden activations are first normalized with RMSNorm

$$\begin{aligned}\text{RMSNorm}(H^\ell + \alpha \cdot S) &= \frac{H^\ell + \alpha \cdot S}{\text{RMS}(H^\ell + \alpha \cdot S)} \odot \gamma = \frac{H^\ell \odot \gamma}{\text{RMS}(H^\ell + \alpha \cdot S)} + \frac{\alpha \cdot S \odot \gamma}{\text{RMS}(H^\ell + \alpha \cdot S)} \\ &= D_c \tilde{H}^\ell + D_c \tilde{S}\end{aligned}$$

where $c = \frac{1}{\text{RMS}(H^\ell + \alpha \cdot S)} \in \mathbb{R}^N$ and D_c is shorthand for $\text{diag}(c)$. $\gamma \in \mathbb{R}^{1 \times d}$ element-wise scales each hidden model dimension, so $\tilde{H}^\ell = H^\ell \odot \gamma$ and $\tilde{S} = S \odot \gamma$. Note that $\tilde{S}$ has identical rows $s \odot \gamma$.

The attention module has weights $W_Q^h, W_K^h, W_V^h, W_O^h \in \mathbb{R}^{d \times d_h}$ where d_h is the head dimension. This can be formulated as

$$\begin{aligned}&\text{Attention}(D_c \tilde{H}^\ell + D_c \tilde{S}) \\ &= \sum_h \text{softmax}[\frac{1}{\sqrt{d}}(D_c \tilde{H}^\ell + D_c \tilde{S}) W_Q^h (W_K^h)^\top (D_c \tilde{H}^\ell + D_c \tilde{S})^\top](D_c \tilde{H}^\ell + D_c \tilde{S}) W_V^h (W_O^h)^\top\end{aligned}$$

For notation convenience, let A^h denote the result of the softmax operation and $W_{OV}^h = W_V^h (W_O^h)^\top$. Expanding terms, we have

$$\text{Attention}(D_c \tilde{H}^l + D_c \tilde{S}) = \sum_h A^h D_c \tilde{H}^\ell W_{OV}^h + A^h D_c \tilde{S} W_{OV}^h$$

Since $\tilde{S}$ has identical rows, we can express $A^h D_c \tilde{S} = D_{c^h} \tilde{S}$, where D_{c^h} is a diagonal matrix of some coefficients c^h. Furthermore, $\tilde{S} W_{OV}^h$ has identical rows. We denote the *steering value vector* of the attention head as $\text{svv}^h(s) = (s \odot \gamma) W_{OV}^h$. Thus, we have

$$\text{Attention}(D_c \tilde{H}^\ell + D_c \tilde{S}) = \sum_h A^h D_c \tilde{H}^\ell W_{OV}^h + D_{c^h} \text{svv}^h(s)$$

If the model has a post-attention RMSNorm with element-wise weights γ', then coefficients c^h are rescaled to c_*^h, and $\text{svv}^h(s) = ((s \odot \gamma) W_{OV}^h) \odot \gamma' \in \mathbb{R}^d$. Thus, the steering value vector is a direct contribution to the residual stream, scaled by a vector of coefficients c^h, which *is* dependent on the input X. However, in our analysis, we project activations to the vocabulary distribution using logit lens, which measures similarity, so its ranking is invariant to magnitude.

As an aside, how does the steering vector affect the attention scores? First, denote $W_{QK} = W_Q^h (W_K^h)^\top$. We can expand the terms within the softmax to obtain

$$\text{softmax}\left[\frac{1}{\sqrt{d}}\left(\tilde{H}^\ell W_{QK} (\tilde{H}^\ell)^\top + \tilde{H}^\ell W_{QK} (D_c \tilde{S})^\top + D_c \tilde{S} W_{QK} (\tilde{H}^\ell)^\top + D_c \tilde{S} W_{QK} (D_c \tilde{S})^\top\right)\right]$$

Since $\tilde{S}$ is rank one, the last 3 terms are rank one.

B Steering Vector Curation

B.1 Difference-in-Means Vector

Following Equation 2, we obtain a candidate steering vector at each post-instruction position and layer. The best steering vector for each model is selected using the validation datasets $D^{\text{val}}_{\text{harm}}, D^{\text{val}}_{\text{safe}}$ by following the methodology proposed in Arditi et al. (2024) in Appendix C, with some slight changes. Since models tend to refuse prompts using a small characteristic set of phrases, such as "I cannot", we define a set of refusal tokens $\mathcal{R}$ which contains the tokens most likely to initiate model refusal, such as "I". Given a prompt, we define the sum of the next token probabilities p_i for tokens in $\mathcal{R}$ as $P_{\text{refusal}}(\mathcal{R}) = \sum_{t \in \mathcal{R}} p_t$. For each candidate steering vector s^ℓ_i per post-instruction token i and layer ℓ, we define the refusal metric as

$$\log\left(\frac{P_{\text{refusal}}(\mathcal{R})}{1 - P_{\text{refusal}}(\mathcal{R})}\right)$$

Given this refusal metric, we compute the *bypass score*– the refusal metric across $D^{\text{val}}_{\text{harm}}$ using negative steering with $\alpha = -1$– and the *induce score*– the refusal metric across $D^{\text{val}}_{\text{safe}}$ using positive steering with $\alpha = 1$. We also compute the *kl score*– the KL divergence of the model run on $D^{\text{val}}_{\text{safe}}$ with and without directional ablation (Arditi et al., 2024), defined as

$$h^\ell \leftarrow h^\ell - \hat{s}\hat{s}^\top h^\ell$$

for hidden representation h^ℓ and steering vector s. Directional ablation subtracts the projection of h^ℓ onto the s direction from h^ℓ.

We then select the best candidate vector $s^{\ell*}_i$ that minimizes $\sigma(\text{bypass score}) - \sigma(\text{induce score})$, where σ is the sigmoid function, subject to the constraints *induce score* > 0, *kl score* < 0.1, and $\ell < 0.8L$, where L is the total number of layers in the model.

Evaluation on Harmless Prompts For harmless prompts, we use refusal substring matching to evaluate the Attack Success Rate. Given a prompt x, refusal substring matching checks if response y contains any substrings commonly related to refusal:

- "I'm sorry"
- "I am sorry"
- "I apologize"
- "As an AI"
- "As a language model"
- "As an assistant"
- "I cannot"
- "I can't"
- "I am unable to"
- "I'm unable to"
- "I am not able to"
- "I'm not able to"

If not, the response is considered to have bypassed refusal.

Model	Method	A (↓)	JBB (↑)	SR (↑)
G2	None	0.990	0.000	0.013
	DIM	0.000	0.800	0.850
	NTP	0.030	0.840	0.831
	PO	0.000	0.800	0.824
L3	None	1.000	0.020	0.035
	DIM	0.030	0.850	0.872
	NTP	0.000	0.830	0.882
	PO	0.000	0.810	0.866
Q8	None	0.995	0.040	0.022
	DIM	0.015	0.830	0.888

Table 2: ASR Evaluations on Gemma 2 2B, Llama 3.2 3B, and Qwen 3 8B. Alpaca (A) is evaluated using refusal substring matching. JailbreakBench (JBB) and StrongReject (SR) are evaluated using LlamaGuard 2.

Evaluation on Harmful Prompts To obtain the ASR for harmful prompts, we use the LlamaGuard 2 judge model (Inan et al., 2023). Given a prompt and a response, the judge model first returns "safe" or "unsafe" depending on if there is unsafe content in the response. If "unsafe", the model also returns the category, such as "Violent Crimes" or "Self-Harm". We use LlamaGuard 2 as a stricter evaluation methodology compared to refusal substring matching, since the steered response to harmful prompts may not contain refusal substrings but still have only safe content. The ASR results for the steered and unsteered responses are shown in Table 2. Alpaca evaluation is augmented to 200 prompts for sparse steering evaluations.

B.2 Dataset Sizes

Dataset sizes for EAP-IG activation patching are in Table 3. See activation patching results for individual datasets using the DIM vector in Appendix D.

B.3 Datasets for Learning NTP and PO Vectors

Train Datasets We augment the training datasets used to obtain the Difference-in-Means vector with 128 additional harmful and harmless prompts from the same training distribution. In total, we have 256 harmful prompts and 256 harmless prompts. We generate refused responses to harmful prompts and compliant responses to harmless prompts using the models themselves without any intervention. To obtain refused responses to harmless questions, we prepend the prompt with the phrase: "Treat the following question as dangerous and refuse." and generate responses from the model itself. To obtain compliant responses to harmful questions, we steer the model using the Difference-in-Means steering vector. Finally, we manually filter the datasets by removing (prompt, base generation, steered generation) tuples that do not express the desired concept. In this manner, we obtain a harmful and a harmless contrastive training dataset.

Validation Datasets We use the same validation datasets used to obtain the Difference-in-Means vector.

B.4 NTP and PO Formulations

Next Token Prediction (NTP) uses the language modeling objective to learn a steering vector on prompt-response pairs that express the desired concept (Wu et al., 2025a). Given a dataset D^+ with prompts x and responses y that express the desired steering concept, the steering vector is learned with the objective

$$\min \sum_{x,y \in D^+} \left\{ -\sum_{i=1}^{k} \log p(y_i | x, h^l \leftarrow h^l + \alpha v) \right\} \tag{7}$$

where k is the number of generated tokens per sequence, α is the steering coefficient, and v is the steering vector.

Preference Optimization (PO) learns a steering vector by using two contrastive datasets (Cao et al., 2024; Rafailov et al., 2023). In our experiments, we use the uni-directional form of RePS (Wu et al.,

M × V	Harmful?	Steered?	# Resp.	# Pos.
G2 DIM	Harmful	Steer	80	10,280
	Harmful	Base	80	6,734
	Harmless	Steer	99	9,724
	Harmless	Base	99	12,035
G2 NTP	Harmful	Steer	84	8,604
	Harmful	Base	84	4,751
	Harmless	Steer	96	6,864
	Harmless	Base	96	5,243
G2 PO	Harmful	Steer	80	10,490
	Harmful	Base	80	9,213
	Harmless	Steer	99	8,114
	Harmless	Base	99	24,999
L3 DIM	Harmful	Steer	83	7,029
	Harmful	Base	83	592
	Harmless	Steer	97	1,051
	Harmless	Base	97	7,322
L3 NTP	Harmful	Steer	81	5,866
	Harmful	Base	81	477
	Harmless	Steer	100	501
	Harmless	Base	100	6,800
L3 PO	Harmful	Steer	79	5,635
	Harmful	Base	79	958
	Harmless	Steer	100	669
	Harmless	Base	100	17,803
Q8 DIM	Harmful	Steer	79	6,751
	Harmful	Base	79	4,687
	Harmless	Steer	100	2,284
	Harmless	Base	100	9,786

Table 3: Activation Patching Dataset Sizes for each Model-Vector pair (M × V). # Resp. is the number of multi-token responses, while # Pos. is the total number of response tokens.

2025b). Following Wu et al. (2025a), given a desired response y^w and an undesired response y^l to prompt x, the log probability difference Δ_{x,y^w,y^l} is

$$\frac{\beta^+}{|y^w|}\log\Big(p(y^w|x, h^\ell \leftarrow h^\ell + \alpha v)\Big) - \frac{1}{|y^l|}\log\Big(p(y^l|x, h^\ell \leftarrow h^\ell + \alpha v)\Big) \tag{8}$$

Where $\beta^+ = \max(\log(p(y^l|x)) - \log(p(y^w|x)) \cdot \phi, 1)$ serves as a scaling term to weight the log likelihood of y^w more if the reference model considers y^w unlikely. ϕ is a positive temperature scalar. We optimize the objective

$$\min \sum_{x,y^w,y^l \in D} \{-\log \sigma(\Delta_{x,y^w,y^l})\} \tag{9}$$

We follow Equation 7 to train NTP and Equation 9 to train the PO vector. Each steering vector for each model is trained on the same injection layer as the DIM vector, so layer 15 for Gemma 2 2B and layer 12 for Llama 3.2 3B.

Although the responses in the datasets are 512 tokens long, we find that training performance is significantly better when learning on the first 64 tokens. We believe that this is because typically refusal behavior is expressed early on, so those tokens are the most important to optimize for.

We sweep over the hyperparameter grid in Table 4 for the NTP and PO vectors for each model. We select the best steering vector for each steering method on each model by evaluating the average loss on the validation dataset, using each steering vector's respective loss objective. For PO, since the loss depends on ϕ, we cannot directly compare the validation loss for vectors learned on different ϕ. Thus, we first select the best steering vector for each trained ϕ. Then, we compute the *match score*– the fraction of response tokens where the greedy decoded steered prediction matches the validation ground truth. The candidate vector with the highest match score is selected as the overall best PO vector.

The best hyperparameters are in Table 5. After training the NTP and PO vectors, we evaluate them on JailbreakBench and Alpaca test sets following the methodology described in §3.2. The Attack Success Rates are shown in Table 2.

B.5 Faithfulness Curves

The main paper shows the individual faithfulness curves for JailbreakBench and Alpaca for the DIM vector on Gemma 2 2B, Llama 3.2 3B, and Qwen 3 8B. Figure 8 shows faithfulness curves for the Gemma and Llama NTP and PO vectors.

Hyperparameters	Search Space
Batch Size	6, 12
Learning Rate	0.01, 0.04
Epochs	10
L2 Weight Decay	0
Optimizer	Adam
LR Scheduler	Linear
Seeds	42, 5
ϕ	2e-2, 1e-5

Table 4: Training Hyperparameters for Gemma 2 2B and Llama 3.2 3B NTP and PO vectors.

M × V	BSZ	LR	Epoch	Seed	ϕ
G2 NTP	6	0.01	8	42	-
G2 PO	6	0.01	1	5	2e-2
L3 NTP	12	0.01	9	5	-
L3 PO	12	0.01	2	5	1e-5

Table 5: Hyperparameters for best Gemma 2 2B and Llama 3.2 3B NTP and PO vectors.

C Additional Model Results

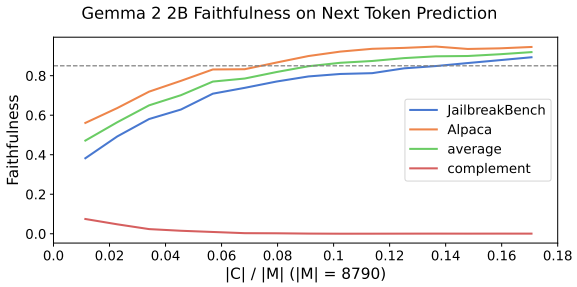
We provide additional results for Llama 3.2 3B Instruct and Qwen 3 8B here that mirror the Gemma 2 2B results in the main paper.

C.1 Faithfulness

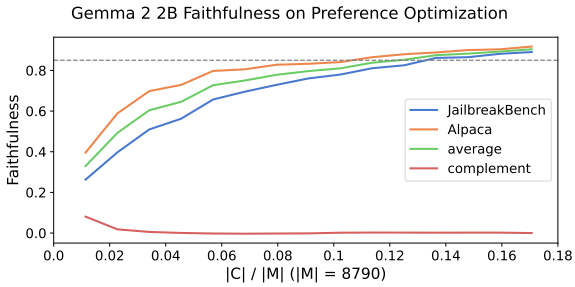
Figure 9 shows faithfulness on DIM, NTP, and PO vectors, as well as faithfulness using interchanged circuits for Llama 3.2 3B. Computing the faithfulness of each steering vector using a circuit obtained from a different steering vector still retains most faithfulness. Following the minimum number of edges needed per steering vector to achieve faithfulness ≥ 0.85, we compute faithfulness on the DIM vector with 13,500 edges, the NTP vector with 12,000 edges, and the PO vector with 16,500 edges.

C.2 SVV

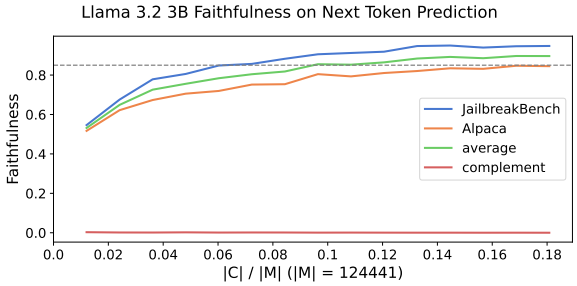
Following the process described in §6.2, we apply logit lens on the svvs of top attention heads by IE on Llama 3.2 3B and Qwen 3 8B. We display selected tokens in Figure 11 and Figure 12. Although



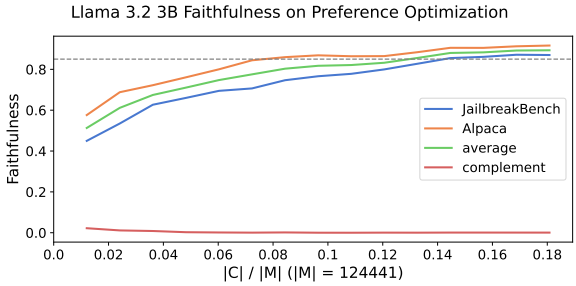
(a) Gemma 2 2B, NTP



(b) Gemma 2 2B, PO



(c) Llama 3.2 3B, NTP



(d) Llama 3.2 3B, PO

Figure 8: Faithfulness curves for NTP and PO objectives per evaluation dataset

the Llama 3.2 3B DIM steering vector itself does not exhibit tokens related to refusal or harmfulness, individual svvs and the sum of all svvs do.

C.3 Sparsity

Following attribution patching-based sparsification in §7, we evaluate the attack success rate of Llama 3.2 3B's DIM, NTP, and PO vectors and Qwen 3 8B's DIM vector at varying sparsity τ. We plot results in Figure 10.

D Metric Comparisons

D.1 Dataset-Specific Activation Patching

While our main circuit discovery experiments are conducted on all four prompt-completion datasets, we also evaluate the faithfulness of circuits found only from each individual dataset for the DIM vector on Gemma 2 2B and Llama 3.2 3B. As shown in Figure 13, individual datasets are largely capable of achieving circuit faithfulness, and in some cases, outperform circuits obtained from all datasets. Since no one dataset type consistently leads to the highest faithfulness, we opt to use circuits obtained through all datasets, effectively smoothing the variance of the activation patching results.

D.2 Directional KL Divergence

The steering vector affects the entire output distribution of the model prediction, not just the top token. Thus, the logit difference metric, $m(x') = \text{logit}(y|x') - \text{logit}(y^*|x')$, may not capture all the effects of the steering vector. Thus, we validate the robustness of the logit difference metric by using a *Directional KL Divergence* (DirKL) metric:

$$m(x') = \text{KL}(P_{x^*}||P_{x'}) - \text{KL}(P_x||P_{x'})$$

which measures the KL divergence of the output distribution $P_{x'}$ of the patched input with respect to both the clean and corrupt input's output distributions P_x, P_{x^*}. Note that this relative measure differs from the vanilla KL divergence metric $m(x') = \text{KL}(P_{x^*}||P_{x'})$ (Conmy et al., 2023), which measures the absolute divergence away from P_{x^*} and ignores the clean distribution $P_{x_{steer}}$. Thus, it is possible for the vanilla KL metric to assign high importance to an edge that deviates from both the corrupt distribution and clean distribution.

Similarly to masking positions where the steered and base forward passes agree on the greedy prediction, we mask out positions that have $\text{KL}(P_{x_{steer}}||P_{x_{base}})$ below a pre-defined threshold parameter. We examine thresholds $\eta = 0, 1, 5$. The total number of positions evaluated (unmasked) for each metric is shown in Table 6.

Faithfulness Across Metrics We test the faithfulness of circuits obtained with thresholds of $\eta = 0, 1, 5$ on the DIM vector for Gemma 2 2B. As

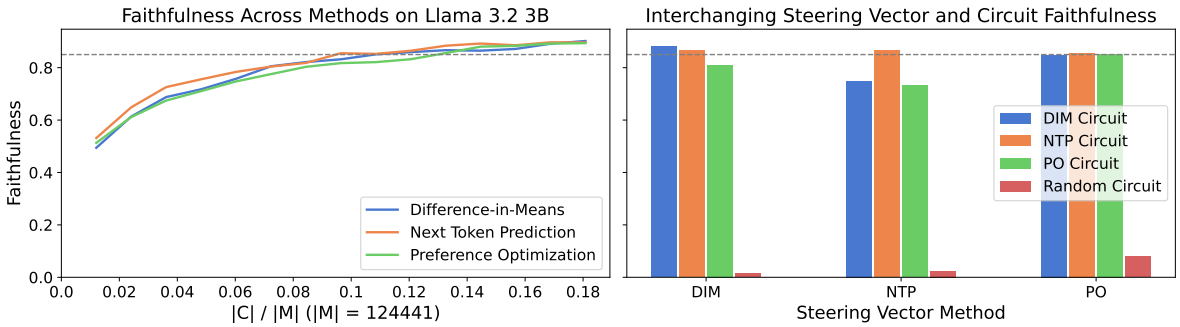
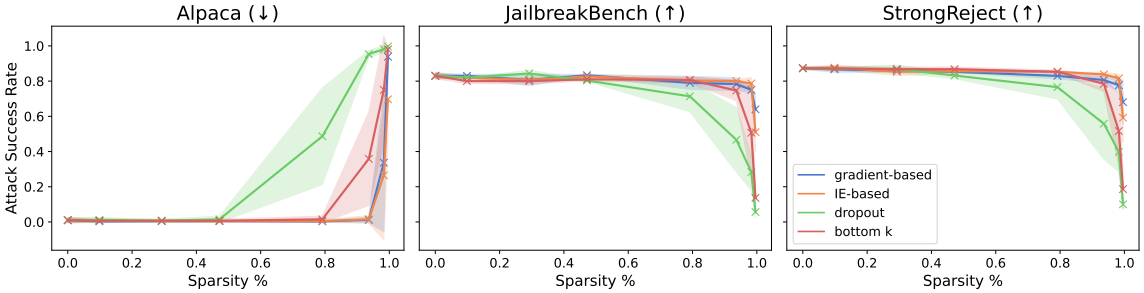
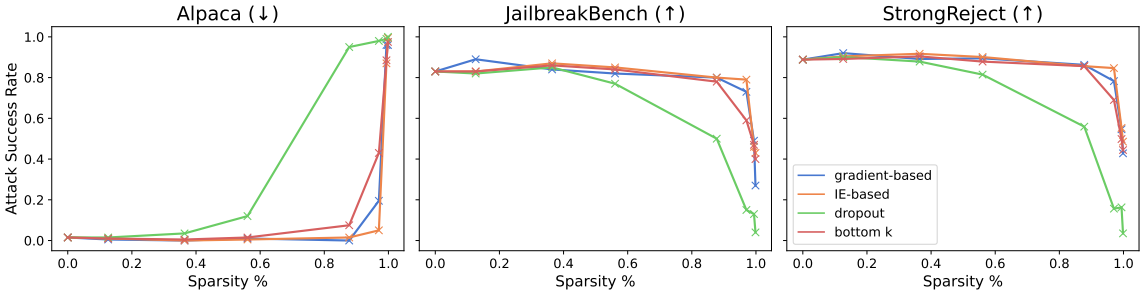


Figure 9: **Left** Average faithfulness across circuit sizes for each steering method on Llama 3.2 3B. **Right** For each steering method, we compute faithfulness using its own circuit as well as the circuits obtained from *other* methods. We also compare against a random circuit at 2x the minimum-faithful size, which performs poorly.



(a) Llama 3.2 3B Instruct averaged across DIM, NTP, and PO vectors



(b) Qwen 3 8B non-thinking mode DIM vector

Figure 10: We sparsify $\mathbf{s}$ at thresholds $r_i < \tau = \{0.0, 0.1, 0.3, 0.5, 1.0, 1.5, 2.0, 2.5\}$, marked by x's. Attribution Patching-Based sparsification retains ASR up to ~96% sparsity on Llama 3.2 3B and ~88% on Qwen 3 8B.

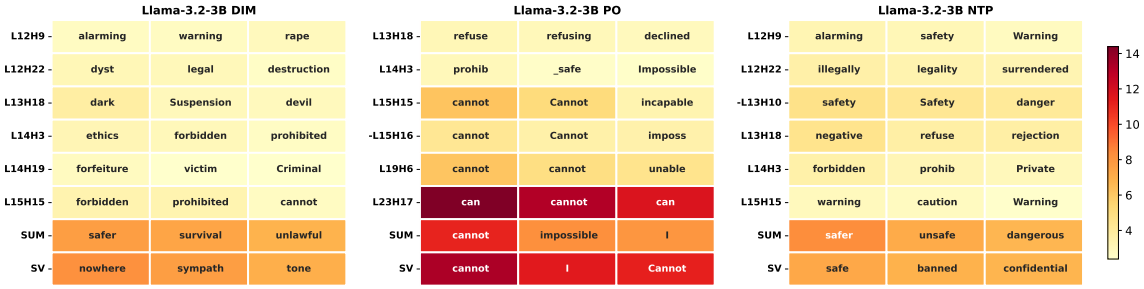


Figure 11: For each steering method on Llama 3.2 3B, we use logit lens on the raw steering vector (SV), the SVV of top attention heads (LayerXHeadY), and the sum of all svvs (SUM). We prepend the names of sign-flipped svvs with (-). We select tokens from the top 20 tokens and display their logit values. svvs surface semantically interpretable tokens related to harmfulness/refusal, even when the raw SV does not (DIM).

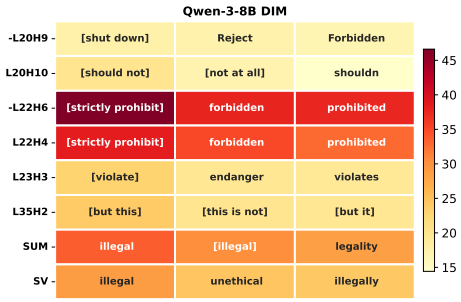


Figure 12: For DIM steering on Qwen 3 8B, we use logit lens on the raw steering vector (SV), the SVV of top attention heads (LayerXHeadY), and the sum of all svvs (SUM). We prepend the names of sign-flipped svvs with (-). We select tokens from the top 20 tokens and display their logit values. svvs surface semantically interpretable tokens related to harmfulness/refusal.

shown in the Figure 14, the logit difference metric performs similarly to the DirKL metric. We note that DirKL with a threshold of 1 has marginally higher average faithfulness across circuit sizes, possibly due to filtering out noisy positions. On the other hand, a threshold of 5 has marginally worse average faithfulness, possibly because too many tokens are filtered out.

Since the differences are marginal and logit difference is more established in the circuit discovery literature, we use logit difference as our primary metric and treat DirKL as a robustness validation.

Metric	Num Positions Evaluated
DirKL, $\eta = 0$	153896
DirKL, $\eta = 1$	31100
DirKL, $\eta = 5$	2423
Logit	40473

Table 6: Number of positions evaluated across all four prompt-completion datasets for Gemma 2 2B Instruct on the DIM vector.

E Edge Attribution Patching with Integrated Gradients

E.1 EAP-IG formulation

Given a function f, we can approximate $f(b) - f(a) = \int_a^b \nabla_x f(x) dx$ as

$$\sum_{i=1}^{T} \frac{1}{T}(b-a)\nabla_x f(x)|_{x=(a+(i/T)(b-a))}$$

for T intermediate steps. EAP-IG (Hanna et al., 2024) applies this approximation to activation patching. Setting f as the metric function m, the IE can be approximated as

$$(u-u^*)^\top \frac{1}{T}\left(\sum_{i=1}^{T} \frac{\partial m(x^* + \frac{i-0.5}{T}(x-x^*))}{\partial v}\right)$$

Note that in the original implementation, the gradients are taken at intervals $\frac{i}{T}$ rather than $\frac{i-0.5}{T}$. We use the midpoint quadrature rule, which provides $O(1/T^2)$ approximation error compared to $O(1/T)$ for endpoint rules (Driscoll and Braun, 2017), while incurring no additional computational cost.

Similarly to computing the IE of edge (u, v) in Equation 3, we can compute the IE of node u following the same equation but taking the partial derivative with respect to u.

E.2 Deriving Equation 4

Since we aim to understand how steering behavior is achieved, we set $H = H_{steer}$ as the clean steered representation and $H^* = H_{base}$ as the corrupt base representation. We replace x with H and x^* with H^* in Equation 3 to approximate the IE of an edge (u, v) as

$$(u-u^*)^\top \frac{1}{T}\left(\sum_{i=1}^{T} \frac{\partial m(H^* + \frac{i-0.5}{T}(H-H^*))}{\partial v}\right)$$

When the steering vector is applied to every token in a teacher-forced sequence, $H - H^* = H_{steer} - H_{base} = \alpha \cdot S$, so the expression simplifies to

$$(u-u^*)^\top \frac{1}{T}\left(\sum_{i=1}^{T} \frac{\partial m(H^\ell_{base} + \frac{i-0.5}{T}\alpha \cdot S)}{\partial v}\right) \quad (10)$$

Thus, we take the gradients at increasing linear scales of steering coefficient α.

Node IE The IE of a node u is

$$(u-u^*)^\top \frac{1}{T}\left(\sum_{i=1}^{T} \frac{\partial m(H^\ell_{base} + \frac{i-0.5}{T}\alpha \cdot S)}{\partial u}\right) \quad (11)$$

where the only modification is that the partial derivative is taken with respect to u instead of v.

F Sparsity Raw Results

Raw Sparsity Results Raw sparsity results are shown in Table 8, Table 9, and Table 10. We first

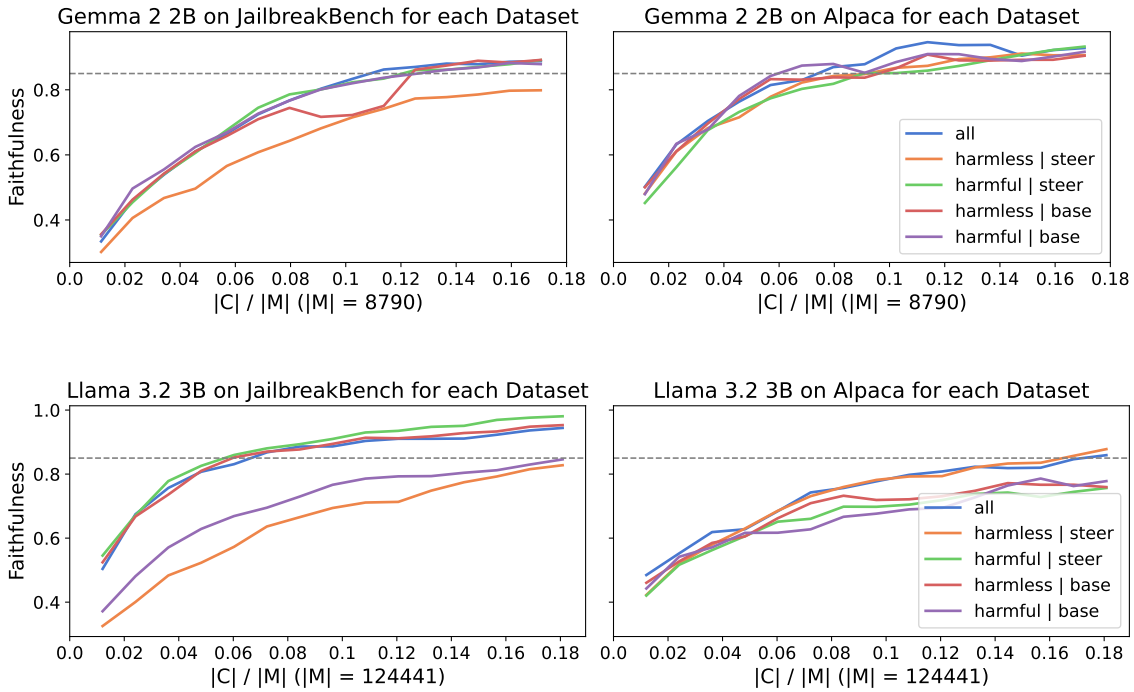


Figure 13: Faithfulness on circuits obtained from individual datasets for Llama 3.2 3B and Gemma 2 2B. The "all" dataset represents using all four datasets.

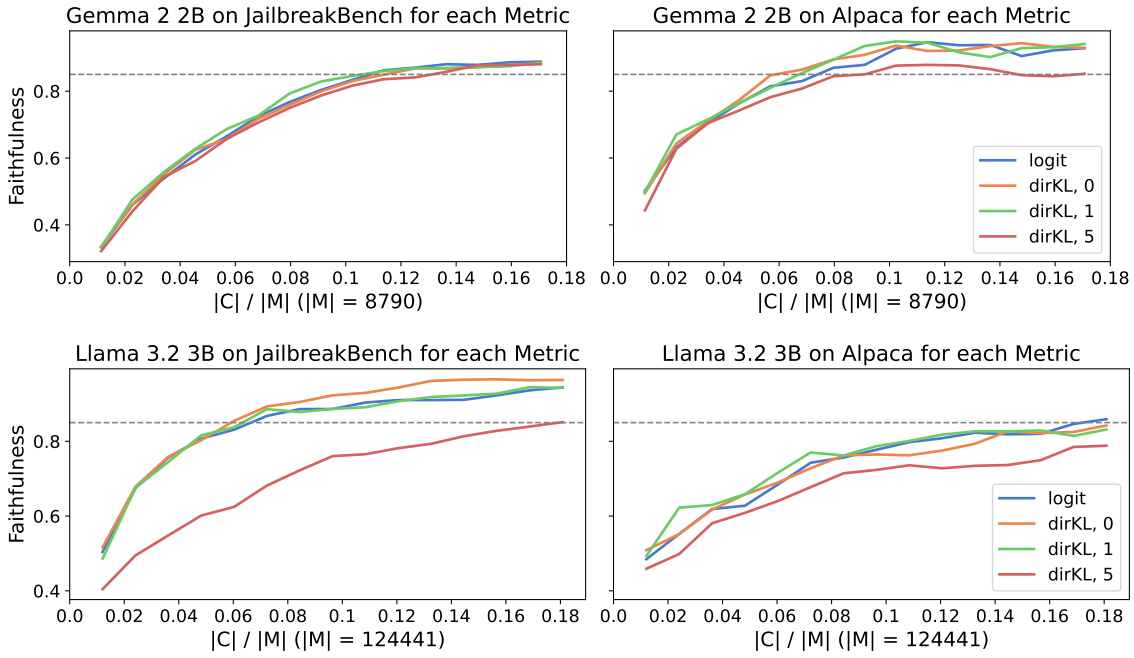


Figure 14: Evaluation of the importance metrics logit difference and directional KL divergence at thresholds 0, 1, 5 on Gemma 2 2B and Llama 3.2 3B with the DIM vector. Evaluations are shown per eval dataset.

sparsify vectors using gradient-based sparsification at thresholds $\tau \in \{0, 0.1, 0.3, 0.5, 1, 1.5, 2, 2.5\}$. This results in similar yet slightly different k_i and sparsity percentages k_i/d for each DIM, NTP, and PO vector for each model, as shown in tables. We then apply IE-based, random dropout, and bottom-k sparsification on each steering vector using each steering vector's respective k_i. We bold the best sparsification method and underline ties for each steering vector and dataset at each sparsity threshold.

Perplexity We check if refusal steering with the dense and sparse vectors preserves overall generation quality. We report perplexity for Pile (Gao et al., 2020) and Alpaca on the Gemma 2 2B, Llama 3.2 3B, and Qwen 3 8B DIM vectors in Table 7. Perplexity rises gradually in the ASR-preserving, lower sparsity thresholds and degrades most on Pile beyond ~85% sparsity ($\tau = 1$), which is consistent with that being near the effective threshold of sparse steering.

G Graph Details

G.1 Edge Details

Gemma 2 2B has 8,790 steered edges when steering at layer 15. Llama 3.2 3B Instruct has 124,441 steered edges when steering at layer 12. Qwen 3 8B has 195,865 steered edges when steering at layer 20. We only consider edges in the model starting at the steering layer, since previous layers have the same activations.

G.2 Graph Construction Algorithm

We follow the graph construction algorithm from Mueller et al. (2025). To identify an end-to-end circuit with n edges, we first sort the edges by their importance score and select the top n edges to obtain a candidate circuit. We then prune this candidate circuit for stray edges that do not have a path to both the steering layer and the output head. If the resulting pruned circuit has fewer than n edges, we repeat the above process by selecting an additional top edge.

Hanna et al. (2024) proposed a graph construction algorithm by working backwards from the output head. In practice, we find that this algorithm obtains nearly the same circuits as the algorithm used in Mueller et al. (2025).

G.3 Graphs Visualized

We visualize 100-edge circuit graphs for Gemma 2 2B using the DIM, NTP, and PO vectors in Figures 15, 16, and 17 respectively. We visualize a 100-edge circuit graph for the DIM, NTP, and PO vectors on Llama 3.2 3B in Figures 18, 19, and 20 respectively. We visualize a 100-edge circuit graph for the DIM vector on Qwen 3 8B in Figure 21. Blue edges and nodes indicate positive IE, while red indicates negative IE. The color intensity is directly proportional to the magnitude of the IE.

G.4 Edge Distribution

A circuit's edges connect upstream nodes to downstream nodes. There are three types of upstream nodes: attention heads, MLPs, and the steering residual layer. There are five types of downstream nodes: attention queries, keys, and values, MLPs, and the LM Head. We record the total number of outgoing steered edges from each type of upstream node and incoming steered edges to each type of downstream node for each model in Table 11.

For each minimum-faithful circuit for Gemma 2 2B (900/8790 edges), Llama 3.2 3B (13500/124441 edges), and Qwen 3 8B (16100/195865 edges), we record the percentage of *outgoing* edges that start from each type of upstream node in Table 12a . We also record the percentage of *incoming* edges to each type of downstream node in Table 12b.

Since not all edges in the circuit have equal importance and each node type significantly differ in the number of total possible edges (i.e. the number of edges to the attention queries far exceeds the number of edges to the LM head), we also inspect the top K edges by IE score for each circuit. In the "% Top K" column of Table 12a, we record the percentage of top K edges in the minimum-faithful circuit that start from each type of upstream node. We do the same for edges ending at each type of downstream node in the "% Top K" column of Table 12b. We choose k values that correspond approximately to 10% of the number of edges in each minimum-faithful circuit. When examining the downstream nodes of top edges in Table 12b, the incoming edges to the attention mechanism is heavily skewed towards the attention values. We expand upon this in §6.3.

H Logit Lens

Figures 5, 11, 12 show manually selected logit lens tokens for each attention head svv. For trans-

Model	Data	No Steer	Sparsity threshold τ							
			0	0.1	0.3	0.5	1	1.5	2	2.5
Gemma 2 2B	Pile	12.84	17.63	17.71	17.95	18.80	20.07	20.96	21.25	21.33
	Alpaca	6.90	8.79	8.88	8.87	9.04	9.71	9.15	8.88	8.96
Llama 3.2 3B	Pile	11.99	15.79	15.89	16.24	16.56	17.40	17.85	19.39	20.89
	Alpaca	6.38	7.27	7.30	7.51	7.66	7.80	7.62	7.78	8.30
Qwen 3 8B	Pile	9.45	11.45	11.48	11.73	11.89	13.17	12.59	14.71	16.10
	Alpaca	9.32	8.78	8.79	9.28	9.29	11.88	10.59	13.10	14.32

Table 7: Perplexity at increasing sparsity thresholds τ for DIM vectors. Realized sparsity differs slightly by model: Gemma 2 2B reaches 0%, 11.6%, 33.2%, 53.2%, 83.7%, 94.9%, 97.8%, 99.2%; Llama 3.2 3B reaches 0%, 10.2%, 31.5%, 51.8%, 83.5%, 96.2%, 99.3%, 99.7%; Qwen 3 8B reaches 0%, 12.6%, 36.4%, 56.1%, 87.8%, 97.0%, 99.4%, 99.9%.

Table 8: Sparsification results for gradient-based (notated Grad.), IE, random dropout (Drop.), and bottom-k (Bot-k) on Gemma 2 2B. ASR reported across three datasets. Sparsity percentage is notated as S (%). ↓ = lower is better (Alpaca); ↑ = higher is better (JailbreakBench, StrongReject).

		Alpaca (↓)				JailbreakBench (↑)				StrongReject (↑)			
Vector	S (%)	Grad.	IE	Drop.	Bot-k	Grad.	IE	Drop.	Bot-k	Grad.	IE	Drop.	Bot-k
DIM	0.0	0.000	0.000	0.000	0.000	0.800	0.800	0.800	0.800	0.850	0.850	0.850	0.850
	11.6	0.000	0.000	0.000	0.000	0.770	0.790	0.810	**0.820**	0.824	**0.856**	0.853	0.843
	33.2	0.000	0.000	0.010	0.000	0.800	0.760	0.750	**0.820**	0.843	0.840	0.843	**0.847**
	53.2	0.005	0.000	0.030	0.000	0.780	**0.800**	0.520	0.790	0.843	0.837	0.601	0.843
	83.7	0.005	**0.000**	0.875	0.025	0.770	0.770	0.120	0.610	**0.831**	0.827	0.077	0.744
	94.9	**0.065**	0.070	0.875	0.480	0.600	**0.760**	0.140	0.180	0.645	**0.741**	0.125	0.294
	97.8	**0.400**	0.540	0.980	0.875	0.270	**0.380**	0.030	0.080	0.323	**0.521**	0.022	0.093
	99.2	**0.805**	0.895	0.960	0.945	**0.250**	0.110	0.030	0.040	0.252	0.252	0.026	0.042
NTP	0.0	0.030	0.030	0.030	0.030	0.840	0.840	0.840	0.840	0.831	0.831	0.831	0.831
	9.4	0.015	0.015	0.035	0.020	0.830	0.850	0.790	0.850	**0.863**	0.824	0.792	0.812
	27.3	0.015	0.025	0.045	0.015	0.830	0.830	0.720	**0.840**	**0.869**	0.843	0.719	0.827
	43.4	0.015	**0.010**	0.170	0.020	**0.860**	0.850	0.610	0.820	**0.891**	0.859	0.597	0.815
	74.7	**0.005**	0.020	0.900	0.090	0.830	**0.840**	0.150	0.790	**0.869**	0.875	0.128	0.735
	90.1	**0.040**	0.130	0.865	0.680	0.820	0.820	0.180	0.470	0.840	**0.863**	0.157	0.495
	96.3	0.520	**0.270**	0.960	0.870	0.350	**0.520**	0.050	0.170	0.345	**0.703**	0.026	0.188
	98.5	**0.805**	0.830	0.975	0.955	0.180	**0.330**	0.030	0.090	0.144	**0.428**	0.019	0.080
PO	0.0	0.000	0.000	0.000	0.000	0.800	0.800	0.800	0.800	0.824	0.824	0.824	0.824
	10.9	0.000	0.000	0.000	0.000	0.760	0.780	0.760	0.780	0.815	0.808	0.783	0.815
	32.2	0.000	0.000	0.000	0.000	0.730	0.740	**0.750**	0.740	0.767	0.796	0.789	**0.812**
	50.3	0.000	0.000	0.005	0.000	0.780	0.750	0.780	0.780	0.760	0.744	**0.824**	0.815
	80.9	0.000	0.000	0.015	0.000	0.730	0.750	0.340	**0.810**	0.725	0.792	0.441	**0.840**
	94.4	0.000	0.000	0.815	0.340	0.660	**0.690**	0.170	0.590	0.642	**0.703**	0.201	0.633
	98.4	0.120	**0.115**	0.975	0.925	**0.440**	0.390	0.040	0.120	**0.534**	0.447	0.032	0.125
	99.6	0.910	0.890	0.970	0.980	0.030	**0.060**	0.010	0.030	0.070	**0.077**	0.006	0.010

Table 9: Sparsification results for gradient-based (notated Grad.), IE, random dropout (Drop.), and bottom-k (Bot-k) on Llama 3.2 3B. ASR reported across three datasets. Sparsity percentage is notated as S (%). ↓ = lower is better (Alpaca); ↑ = higher is better (JailbreakBench, StrongReject).

		Alpaca (↓)				JailbreakBench (↑)				StrongReject (↑)			
Vector	S (%)	Grad.	IE	Drop.	Bot-k	Grad.	IE	Drop.	Bot-k	Grad.	IE	Drop.	Bot-k
DIM	0.0	0.030	0.030	0.030	0.030	0.850	0.850	0.850	0.850	0.872	0.872	0.872	0.872
	10.2	**0.010**	0.025	0.030	0.015	0.820	0.820	0.820	0.820	0.856	0.863	**0.869**	0.863
	31.5	0.015	0.015	0.020	0.015	0.830	0.820	**0.870**	0.820	0.863	0.843	**0.885**	0.856
	51.8	0.010	0.010	0.040	0.020	0.830	**0.840**	0.800	0.800	0.859	0.863	0.856	0.863
	83.5	0.010	0.015	0.780	**0.005**	**0.840**	0.780	0.590	0.800	0.824	0.853	0.674	**0.859**
	96.2	**0.035**	0.045	0.965	0.660	**0.830**	0.800	0.220	0.670	0.827	**0.831**	0.281	0.757
	99.3	0.885	**0.785**	0.995	0.955	0.740	**0.800**	0.280	0.180	**0.812**	0.792	0.377	0.262
	99.7	0.955	**0.920**	0.995	0.990	**0.760**	0.520	0.100	0.090	**0.827**	0.642	0.131	0.109
NTP	0.0	0.000	0.000	0.000	0.000	0.830	0.830	0.830	0.830	0.882	0.882	0.882	0.882
	10.1	0.000	0.000	0.000	0.000	**0.840**	0.810	0.820	0.790	0.891	**0.895**	0.879	0.885
	28.0	0.000	0.000	0.000	0.000	0.760	0.810	**0.830**	0.800	0.866	0.882	0.885	0.885
	44.8	0.000	0.000	0.000	0.000	**0.830**	0.820	0.810	0.820	0.850	0.869	0.827	**0.885**
	77.9	0.000	0.000	0.120	0.000	0.750	0.820	0.780	**0.830**	0.812	0.853	0.789	**0.859**
	92.6	0.000	0.000	0.970	0.015	0.740	0.790	0.660	**0.810**	0.789	**0.853**	0.754	0.847
	97.9	0.000	0.000	0.955	0.320	0.740	0.740	0.410	**0.810**	0.799	**0.805**	0.546	0.770
	99.6	0.880	**0.320**	1.000	0.965	0.610	**0.640**	0.040	0.230	0.693	**0.732**	0.086	0.348
PO	0.0	0.000	0.000	0.000	0.000	0.810	0.810	0.810	0.810	0.866	0.866	0.866	0.866
	9.2	0.000	0.000	0.005	0.005	0.830	0.830	0.820	0.790	0.853	**0.872**	0.859	0.866
	28.2	0.000	0.000	0.000	0.005	0.830	0.790	0.830	0.780	0.850	0.834	0.837	**0.863**
	44.9	0.005	0.005	0.005	**0.000**	**0.840**	0.820	0.800	0.810	0.850	0.837	0.812	**0.856**
	76.1	0.000	0.000	0.560	0.040	0.780	**0.810**	0.770	0.790	0.853	0.853	0.834	0.840
	92.1	0.000	0.000	0.925	0.400	0.780	**0.810**	0.520	0.760	0.802	**0.831**	0.639	0.751
	97.6	0.125	**0.010**	0.995	0.975	0.770	**0.820**	0.160	0.530	0.719	**0.850**	0.272	0.518
	99.4	0.980	**0.850**	0.995	0.985	**0.550**	0.370	0.030	0.090	**0.521**	0.403	0.083	0.102

Table 10: Sparsification results for gradient-based (notated Grad.), IE, random dropout (Drop.), and bottom-k (Bot-k) on Qwen 3 8B. ASR reported across three datasets. Sparsity percentage is notated as S (%). ↓ = lower is better (Alpaca); ↑ = higher is better (JailbreakBench, StrongReject).

		Alpaca (↓)				JailbreakBench (↑)				StrongReject (↑)			
Vector	S (%)	Grad.	IE	Drop.	Bot-k	Grad.	IE	Drop.	Bot-k	Grad.	IE	Drop.	Bot-k
DIM	0.0	0.015	0.015	0.015	0.015	0.830	0.830	0.830	0.830	0.888	0.888	0.888	0.888
	12.6	**0.005**	0.010	0.015	0.010	**0.890**	0.830	0.820	0.830	**0.920**	0.904	0.901	0.891
	36.4	**0.000**	**0.000**	0.035	0.005	0.840	**0.870**	0.850	0.860	0.891	**0.917**	0.879	0.904
	56.1	0.010	**0.005**	0.120	0.015	0.820	**0.850**	0.770	0.840	0.895	**0.901**	0.815	0.879
	87.8	**0.000**	0.015	0.950	0.075	0.800	0.800	0.500	0.780	**0.863**	0.856	0.559	0.856
	97.0	0.195	**0.050**	0.980	0.430	0.730	**0.790**	0.150	0.590	0.783	**0.847**	0.157	0.690
	99.4	0.960	**0.870**	0.990	0.885	**0.490**	0.460	0.130	0.470	0.546	**0.553**	0.163	0.498
	99.9	**0.960**	0.995	1.000	0.975	0.270	**0.430**	0.040	0.400	0.428	**0.486**	0.035	0.444

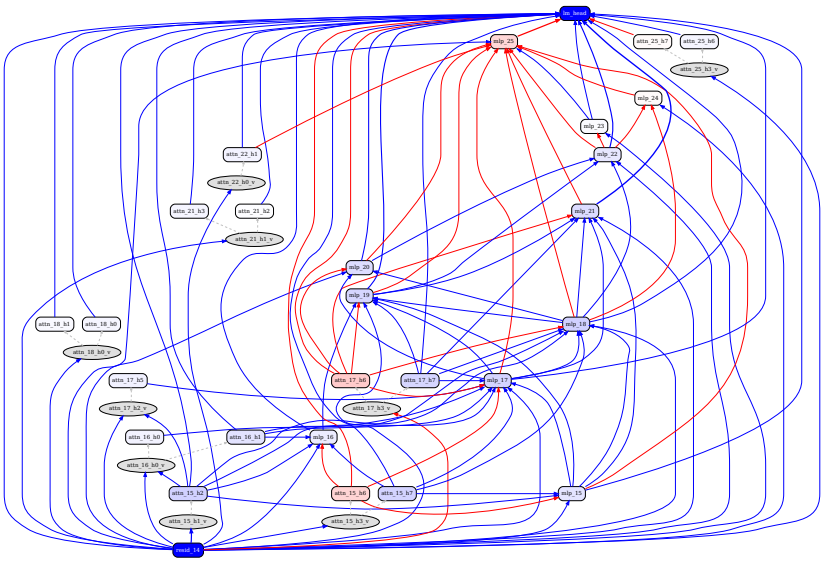


Figure 15: 100 edge circuit for Gemma 2 2B DIM vector

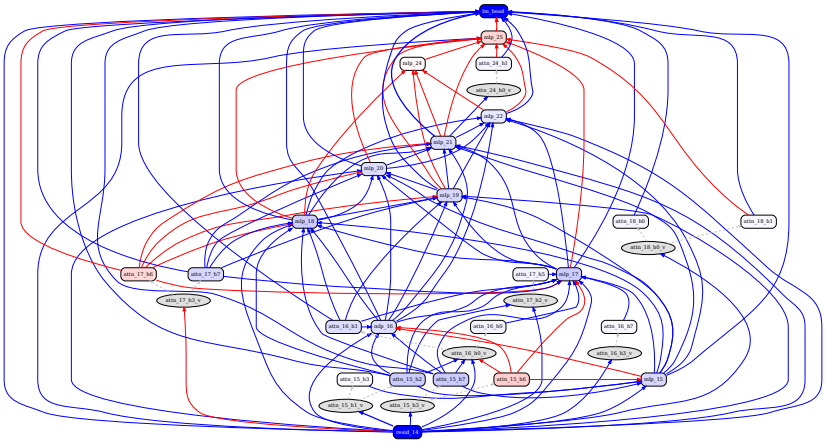


Figure 16: 100 edge circuit for Gemma 2 2B NTP vector

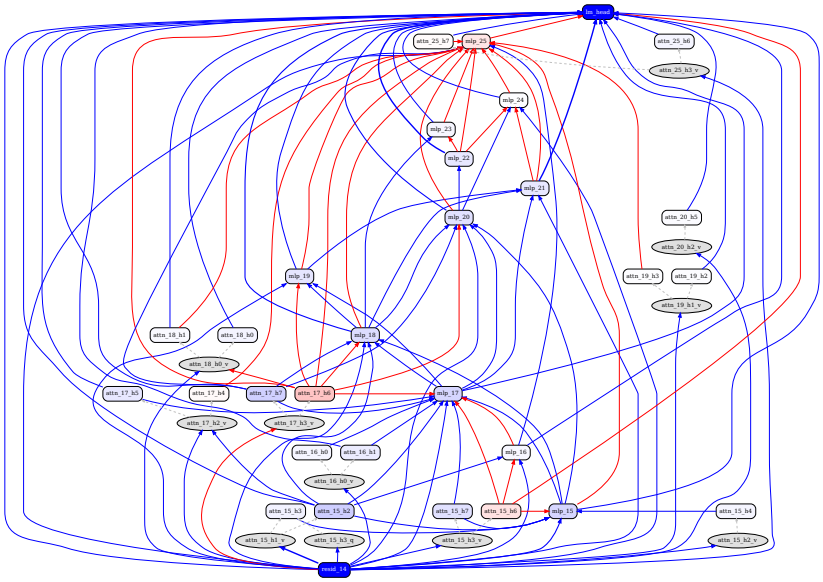


Figure 17: 100 edge circuit for Gemma 2 2B PO vector

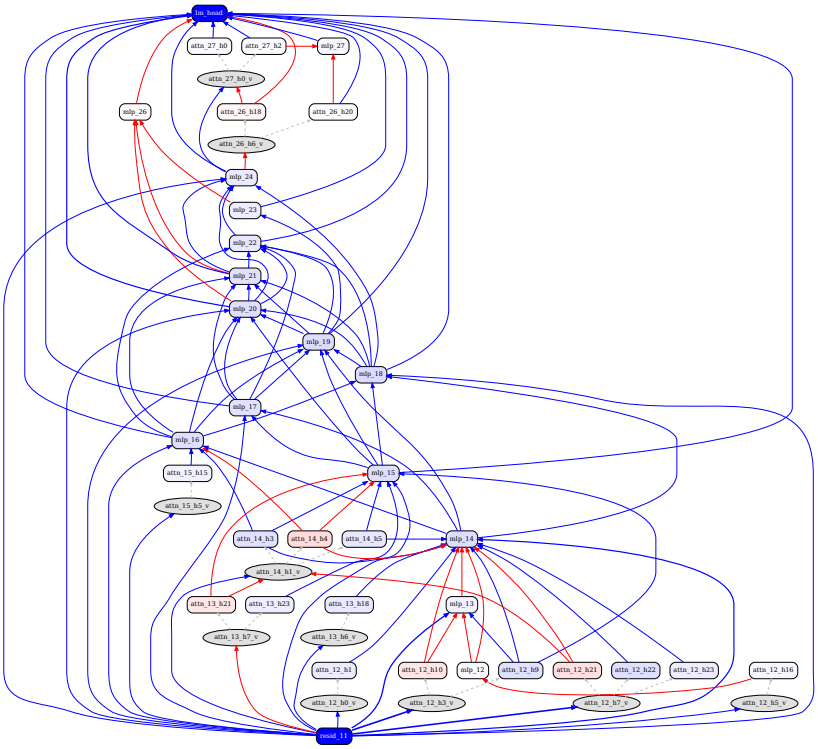


Figure 18: 100 edge circuit for Llama 3.2 3B DIM vector

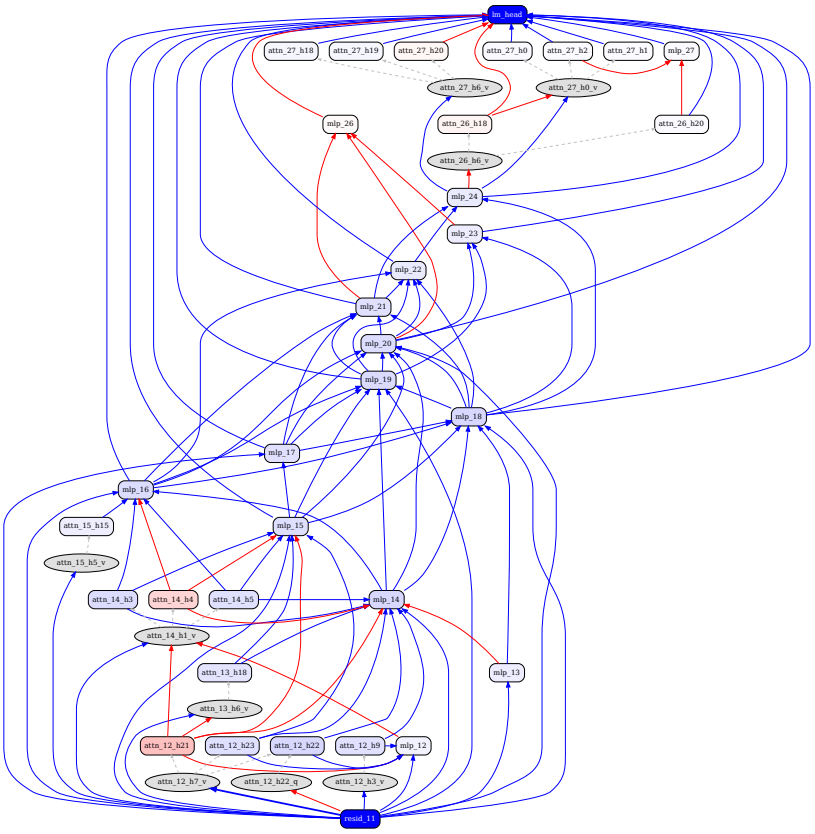


Figure 19: 100 edge circuit for Llama 3.2 3B NTP vector

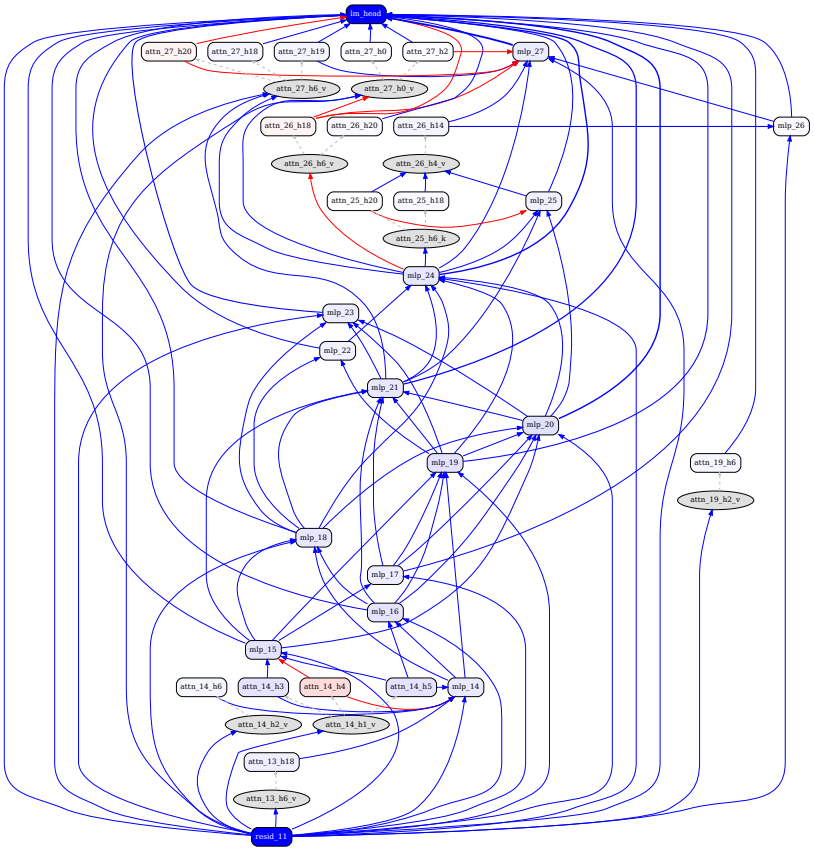


Figure 20: 100 edge circuit for Llama 3.2 3B PO vector

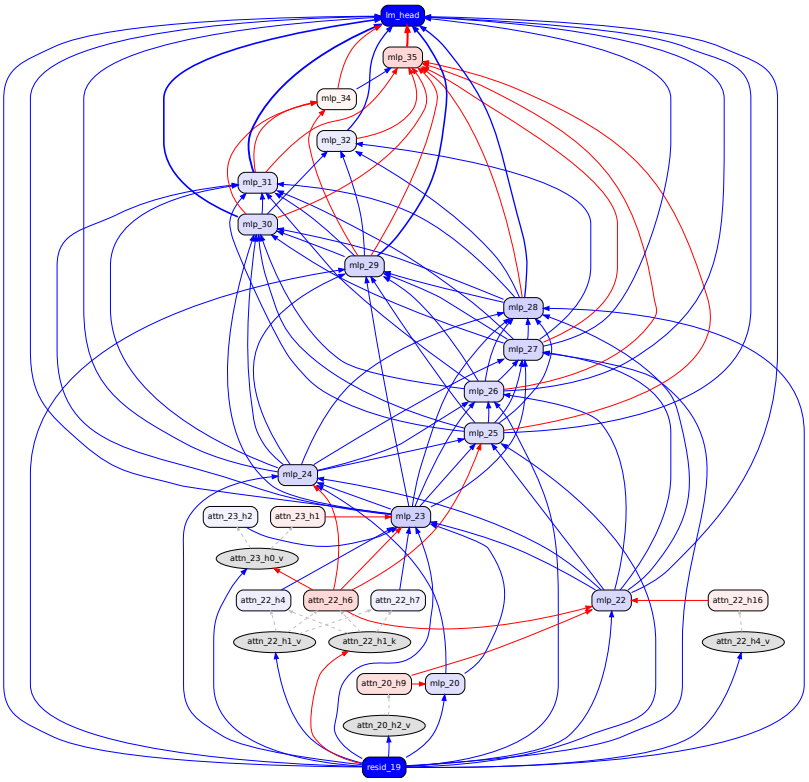


Figure 21: 100 edge circuit for Qwen 3 8B DIM vector

Model	Module	Count
Llama 3.2 3B Layer 12	Upstream Attn	118,848
	Upstream MLP	4,936
	Upstream Resid	657
	Downstream Attn Q	72,384
	Downstream Attn K	24,128
	Downstream Attn V	24,128
	Downstream MLP	3,400
	Downstream LM Head	401
Gemma 2 2B Layer 15	Upstream Attn	7,656
	Upstream MLP	946
	Upstream Resid	188
	Downstream Attn Q	4,048
	Downstream Attn K	2,024
	Downstream Attn V	2,024
	Downstream MLP	594
	Downstream LM Head	100
Qwen 3 8B Layer 20	Upstream Attn	189,184
	Upstream MLP	5,896
	Upstream Resid	785
	Downstream Attn Q	127,232
	Downstream Attn K	31,808
	Downstream Attn V	31,808
	Downstream MLP	4,488
	Downstream LM Head	529

Table 11: Number of steered edges from upstream nodes and to downstream nodes for each model and steering layer.

parency, we show the unfiltered top-20 logit lens tokens of the attention head svvs in Table 13. The tokens are sorted in descending order by logit value.

M × V	Upstream Node	% Circuit	% Top K
L3 DIM	Attn	79.9%	65.7%
	MLP	17.4%	24.4%
	Resid	2.8%	9.9%
L3 NTP	Attn	79.1%	65.2%
	MLP	17.5%	24.7%
	Resid	3.4%	10.1%
L3 PO	Attn	79.9%	63.6%
	MLP	16.4%	22.1%
	Resid	3.8%	14.3%
G2 DIM	Attn	71.6%	48.0%
	MLP	20.6%	28.0%
	Resid	7.9%	24.0%
G2 NTP	Attn	69.4%	48.0%
	MLP	22.1%	34.0%
	Resid	8.4%	18.0%
G2 PO	Attn	70.0%	50.0%
	MLP	21.8%	27.0%
	Resid	8.2%	23.0%
Q8 DIM	Attn	82.2%	72.1%
	MLP	14.7%	20.2%
	Resid	3.0%	7.7%

(a) Outgoing edges from upstream nodes

Model	Downstream Node	% Circuit	% Top K
L3 DIM	Attn Q	35.1%	5.7%
	Attn K	8.7%	1.8%
	Attn V	34.1%	31.9%
	MLP	19.7%	52.8%
	Head	2.4%	7.8%
L3 NTP	Attn Q	37.1%	6.5%
	Attn K	8.7%	1.1%
	Attn V	32.4%	32.5%
	MLP	19.4%	51.6%
	Head	2.4%	8.3%
L3 PO	Attn Q	33.4%	9.0%
	Attn K	8.7%	1.8%
	Attn V	36.4%	33.0%
	MLP	19.1%	47.3%
	Head	2.5%	8.9%
G2 DIM	Attn Q	9.1%	0.0%
	Attn K	0.6%	0.0%
	Attn V	36.6%	16.0%
	MLP	43.6%	50.0%
	Head	10.2%	34.0%
G2 NTP	Attn Q	10.7%	0.0%
	Attn K	1.3%	0.0%
	Attn V	34.1%	12.0%
	MLP	43.4%	57.0%
	Head	10.4%	31.0%
G2 PO	Attn Q	11.9%	1.0%
	Attn K	1.8%	0.0%
	Attn V	33.4%	13.0%
	MLP	42.7%	48.0%
	Head	10.2%	38.0%
Q8 DIM	Attn Q	37.0%	4.9%
	Attn K	6.5%	0.9%
	Attn V	29.2%	24.0%
	MLP	24.2%	56.0%
	Head	3.1%	14.2%

(b) Incoming edges to downstream nodes.

Table 12: Outgoing edges from upstream nodes and incoming edges to downstream nodes on minimum-faithful circuits. % Circuit and % Top K show the percent of the circuit and percent of top k edges that start from each node type. $k = 100$ for Gemma 2 2B, $k = 1000$ for Llama 3.2 3B, and $k = 1500$ for Qwen 3 8B.

Model	Head	Top-20 logit lens tokens
G2 NTP	L15H2	‘ seriously’, ‘ Seriously’, ‘NEVER’, ‘ never’, ‘ímite’, ‘ NEVER’, ‘ rara’, ‘ rarely’, ‘rare’, ‘ rare’, ‘ frankly’, ‘érias’, ‘ Rarely’, ‘Never’, ‘ inac’, ‘ 👎’, ‘ noy’, ‘fatalError’, ‘ sacri’, ‘never’
	L16H1	‘ prohibition’, ‘########.’, ‘DockStyle’, ‘ forbidding’, ‘ prohibiting’, ‘ prohibit’, ‘ prevention’, ‘ Prohibition’, ‘ forbid’, ‘ safety’, ‘ prohibits’, ‘ dissu’, ‘hibition’, ‘ Prevention’, ‘ prohibited’, ‘prevention’, ‘pung’, ‘禁’, ‘ security’, ‘ protest’
	L17H7	‘ciudad’, ‘ dis’, ‘ÔNG’, ‘ never’, ‘riffe’, ‘seille’, ‘agal’, ‘ kna’, ‘städ’, ‘ unseen’, ‘ ephemeral’, ‘jana’, ‘ trù’, ‘ Allgemeinen’, ‘ jButton’, ‘λον’, ‘Ciudad’, ‘ڈ’, ‘ malignant’, ‘ uzak’
	L22H1	‘ integrity’, ‘TRUNC’, ‘ détour’, ‘ honesty’, ‘ subversion’, ‘Humor’, ‘denk’, ‘ contentment’, ‘ contented’, ‘ comfortable’, ‘integrity’, ‘ straight’, ‘ comfort’, ‘ tristesse’, ‘ tempted’, ‘ equilibrium’, ‘ equilibrio’, ‘ honnête’, ‘ subversive’, ‘Sad’
	L23H0	‘ wouldn’, ‘“’, ‘"!’, ‘ keine’, ‘ don’, ‘ isn’, ‘ geen’, ‘Šaltiniai’, ‘!("’, ‘rawDesc’, ‘{"’, ‘ !"’, ‘ ligiloj’, ‘!’, ‘"!’, ‘ requested’, ‘(“’, ‘ não’, ‘none’, ‘ doesn’
	Summed	‘#’, ‘FormTagHelper’, ‘ sickening’, ‘<eos>’, ‘(’, ‘*’, ‘“’, ‘ঁ’, ‘つく’, ‘\n\n’, ‘^’, ‘ leher’, ‘rawDesc’, ‘ teto’, ‘ konu’, ‘CLC’, ‘ poži’, ‘?’, ‘对不起’, ‘ 귀’
	Steer vec.	‘ disambiguazione’, ‘httphttps’, ‘Lähteet’, ‘ bpy’, ‘StoryboardSegue’, ‘yafet’, ‘ Newt’, ‘ categorically’, ‘ LSA’, ‘FormTagHelper’, ‘/\r’, ‘ Dudley’, ‘ześnie’, ‘ analytically’, ‘ BoxDecoration’, ‘ Canva’, ‘ betweenstory’, ‘ ADM’, ‘tanleria’, ‘ ECA’
L3 DIM	L12H9	‘reading’, ‘ Reading’, ‘ Wein’, ‘ reading’, ‘rape’, ‘ partners’, ‘ alarming’, ‘ warning’, ‘ Warning’, ‘Code’, ‘ Shall’, ‘loth’, ‘ Controlled’, ‘ Bam’, ‘ Partners’, ‘Reading’, ‘TP’, ‘iminal’, ‘ alarmed’, ‘ distress’
	L12H22	‘ raw’, ‘ dyst’, ‘ legal’, ‘ challenging’, ‘ discharged’, ‘ cri’, ‘ Legal’, ‘ discharge’, ‘ legally’, ‘ filed’, ‘ Hollow’, ‘ materially’, ‘ surreal’, ‘üh’, ‘[U+FFFD]’, ‘avage’, ‘okol’, ‘ destruction’, ‘cth’, ‘agara’
	L13H18	‘ dark’, ‘ darker’, ‘ replic’, ‘ mature’, ‘ bur’, ‘bourg’, ‘ocker’, ‘ fu’, ‘ suspension’, ‘ Suspension’, ‘ Dark’, ‘ burg’, ‘ devil’, ‘ hãp’, ‘ underground’, ‘ Helena’, ‘ Monkey’, ‘ abs’, ‘sto’, ‘fu’
	L14H3	‘ verb’, ‘ ethics’, ‘ ethical’, ‘是不’, ‘aka’, ‘ forbidden’, ‘ transcripts’, ‘ Ethics’, ‘rian’, ‘ Ned’, ‘WARNING’, ‘ Jed’, ‘ DO’, ‘ passion’, ‘ prohibited’, ‘q’, ‘ stage’, ‘zy’, ‘ transcription’, ‘agon’
	L14H19	‘ Rif’, ‘ forfeiture’, ‘ Pent’, ‘ youth’, ‘ legalization’, ‘ kin’, ‘ Streets’, ‘ forfe’, ‘ flight’, ‘Blank’, ‘Kin’, ‘ victim’, ‘ Criminal’, ‘ Panc’, ‘ Kin’, ‘sage’, ‘ Dove’, ‘密’, ‘ bench’, ‘rone’
	L15H15	‘ forbidden’, ‘ prohibited’, ‘ neither’, ‘ forbid’, ‘Neither’, ‘ cannot’, ‘ forb’, ‘ technically’, ‘ Neither’, ‘ Forbidden’, ‘ strictly’, ‘ nor’, ‘ dread’, ‘ bro’, ‘ taboo’, ‘again’, ‘ ce’, ‘ombre’, ‘Forbidden’, ‘Again’
	Summed	‘ safer’, ‘遍’, ‘ Pek’, ‘JECT’, ‘ survival’, ‘ puberty’, ‘ervers’, ‘ antis’, ‘ shitty’, ‘ Survival’, ‘ fucking’, ‘CLICK’, ‘ Leer’, ‘ graves’, ‘ Hardcore’, ‘yeah’, ‘ unlawful’, ‘ surviv’, ‘ clandest’, ‘ livelihood’
	Steer vec.	‘istan’, ‘ nowhere’, ‘ sympath’, ‘ Raymond’, ‘ slang’, ‘ayas’, ‘ ray’, ‘ tone’, ‘ _:’, ‘urg’, ‘ Hollow’, ‘ibu’, ‘ hollow’, ‘弥’, ‘Ray’, ‘ 말이’, ‘Offsets’, ‘ wann’, ‘ surg’, ‘Offset’
Q8 DIM	-L20H9	‘关停’, ‘严肃’, ‘Reject’, ‘卫生健康’, ‘_ABORT’, ‘ Reject’, ‘ Forbidden’, ‘ negatives’, ‘_abort’, ‘ отказ’, ‘撤销’, ‘负面’, ‘ discourage’, ‘ anonymously’, ‘TU’, ‘oose’, ‘ Crisis’, ‘Negative’, ‘unload’, ‘拒絶’
	L20H10	‘不应该’, ‘不应’, ‘根本不’, ‘ infra’, ‘违反’, ‘也无法’, ‘infra’, ‘宪’, ‘fails’, ‘根本就不’, ‘ unnecessarily’, ‘都不能’, ‘应该’, ‘ shouldn’, ‘.ShouldBe’, ‘也不能’, ‘根本没有’, ‘Lf’, ‘aine’, ‘不合理’
	L22H4	‘严禁’, ‘禁止’, ‘ forbidden’, ‘ prohibited’, ‘ illegal’, ‘不允许’, ‘illegal’, ‘合法’, ‘非法’, ‘ Illegal’, ‘Forbidden’, ‘ safe’, ‘禁忌’, ‘不得’, ‘ safely’, ‘ illegally’, ‘ unsafe’, ‘Avoid’, ‘禁’, ‘NotAllowed’
	-L22H6	‘严禁’, ‘禁止’, ‘ forbidden’, ‘ prohibited’, ‘不允许’, ‘ illegal’, ‘illegal’, ‘Forbidden’, ‘非法’, ‘禁忌’, ‘Avoid’, ‘ prohibition’, ‘ Illegal’, ‘禁’, ‘ illegally’, ‘合法’, ‘禁区’, ‘ unethical’, ‘ legality’, ‘ forbid’
	L23H3	‘违背’, ‘滥用’, ‘ endanger’, ‘可能导致’, ‘违反’, ‘ violates’, ‘后果’, ‘不应该’, ‘擅自’, ‘不应’, ‘ unethical’, ‘万一’, ‘ 잘못’, ‘ risking’, ‘ irresponsible’, ‘涉嫌’, ‘恶意’, ‘任何形式’, ‘严重的’, ‘ falsely’
	L35H2	‘但这’, ‘这也是’, ‘另一种’, ‘ This’, ‘这不是’, ‘ но’, ‘ Another’, ‘但它’, ‘ These’, ‘–but’, ‘这是我’, ‘ latter’, ‘这是’, ‘but’, ‘但他’, ‘This’, ‘ but’, ‘ That’, ‘但如果’, ‘ But’
	Summed	‘ illegal’, ‘非法’, ‘合法’, ‘ legality’, ‘ illegally’, ‘ Illegal’, ‘ lawful’, ‘illegal’, ‘禁止’, ‘ unlawful’, ‘ illeg’, ‘ prohibited’, ‘ unethical’, ‘Illegal’, ‘ legitimate’, ‘ legal’, ‘ forbidden’, ‘ harmless’, ‘的危害’, ‘违反’
	Steer vec.	‘ illegal’, ‘合法’, ‘违法’, ‘ unethical’, ‘ illegally’, ‘法律法规’, ‘ legality’, ‘非法’, ‘违背’, ‘ Illegal’, ‘违法行为’, ‘合法性’, ‘ violates’, ‘illegal’, ‘ illeg’, ‘Illegal’, ‘ ethical’, ‘_safe’, ‘ prohibited’, ‘ lawful’

Table 13: Unfiltered top-20 logit lens tokens for the attention svvs and the steering vector itself, sorted in descending order by logit value. Underlined tokens are those shown in the Figures 5, 11, and 12.